

模式识别与计算机视觉

2D图像变换

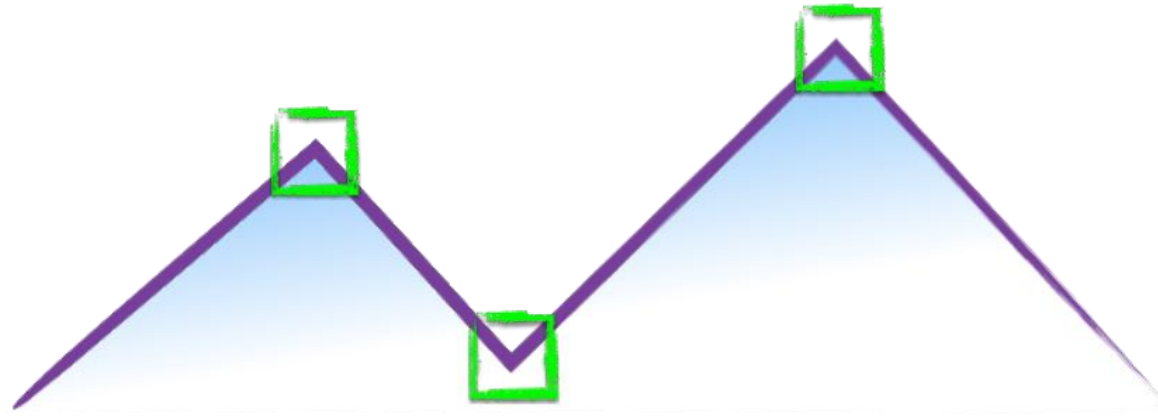
张振宇

南京大学智能科学与技术学院

2025

How do you find a corner?

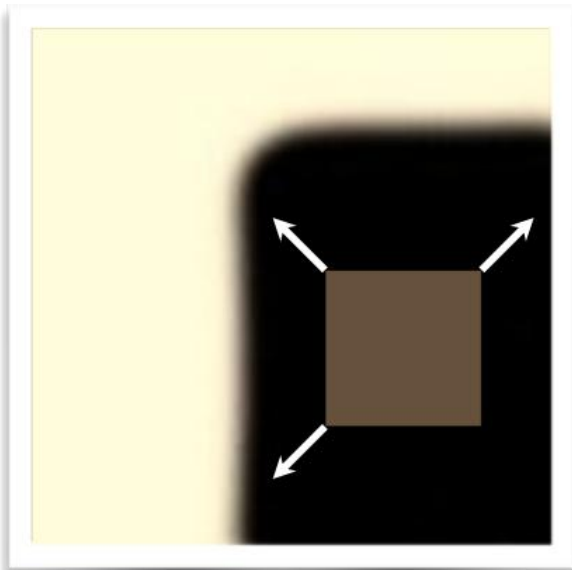
[Moravec 1980]



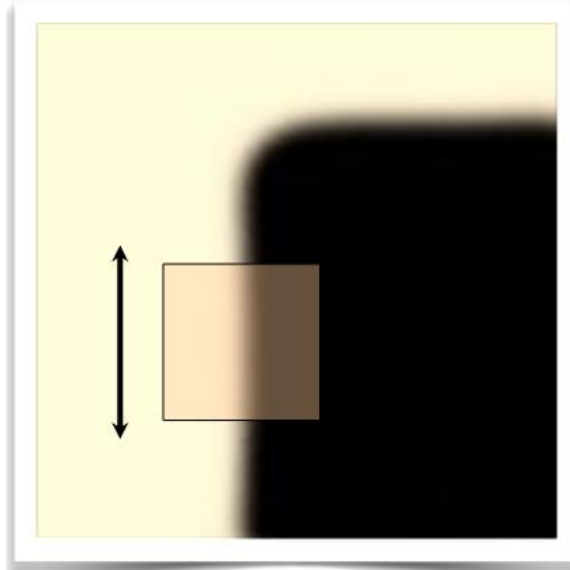
Easily recognized by looking through a small window

Shifting the window should give large change in intensity

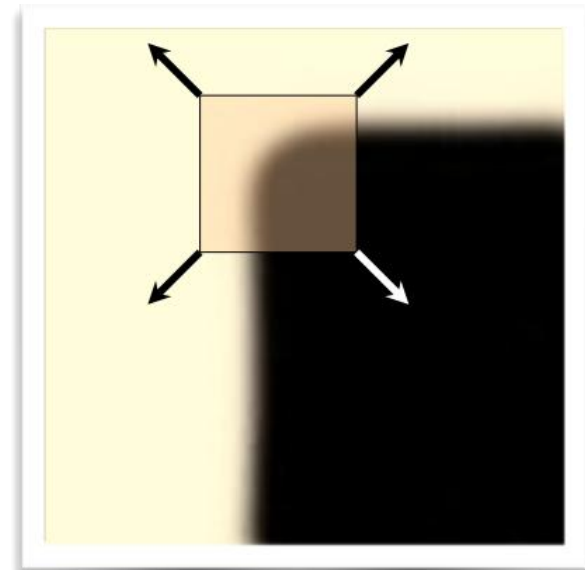
Easily recognized by looking through a small window
Shifting the window should give large change in intensity



“flat” region:
no change in all
directions



“edge”:
no change along the edge
direction



“corner”:
significant change in all
directions

Finding corners

(a.k.a. PCA)

1. Compute image gradients over small region
2. Subtract mean from each image gradient
3. Compute the covariance matrix
4. Compute eigenvectors and eigenvalues
5. Use threshold on eigenvalues to detect corners

$$I_x = \frac{\partial I}{\partial x}$$

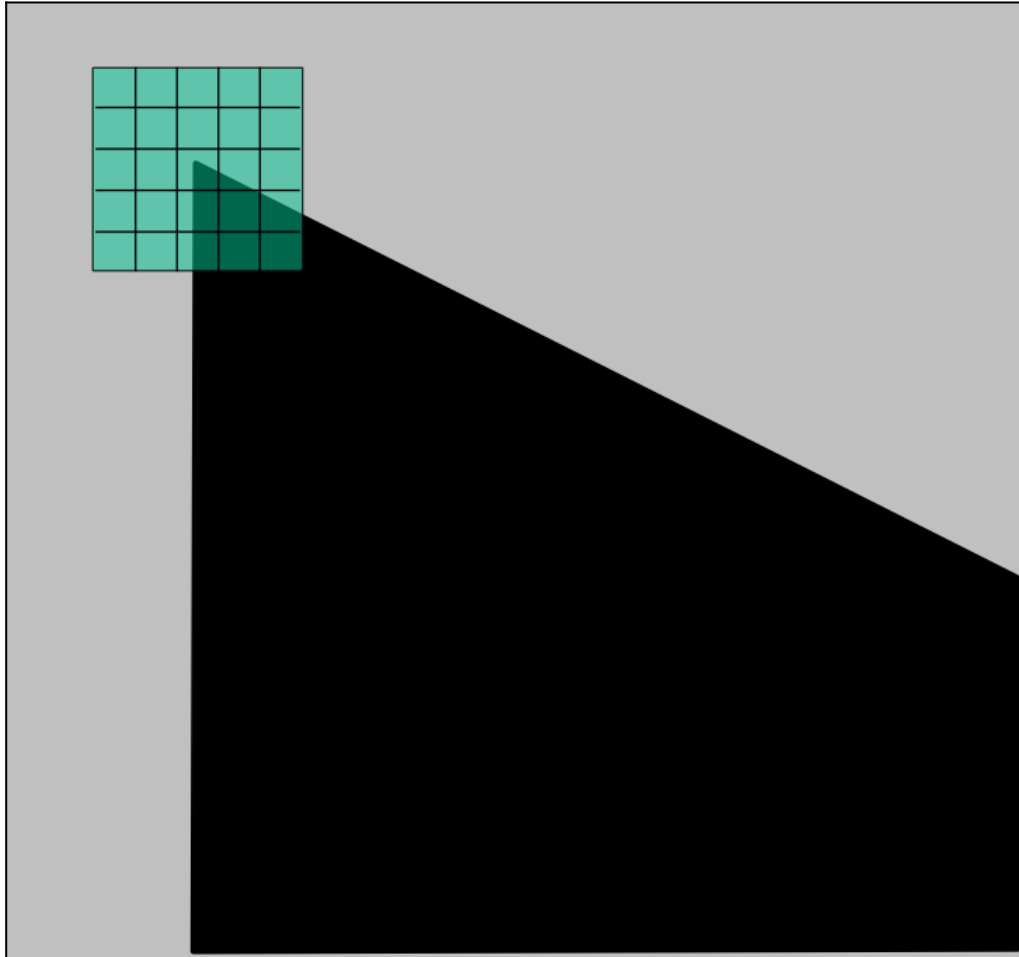


$$I_y = \frac{\partial I}{\partial y}$$



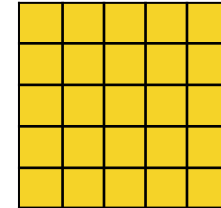
$$\begin{bmatrix} \sum_{p \in P} I_x I_x & \sum_{p \in P} I_x I_y \\ \sum_{p \in P} I_y I_x & \sum_{p \in P} I_y I_y \end{bmatrix}$$

1. Compute image gradients over a small region (not just a single pixel)



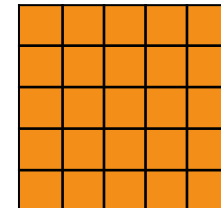
array of x gradients

$$I_x = \frac{\partial I}{\partial x}$$

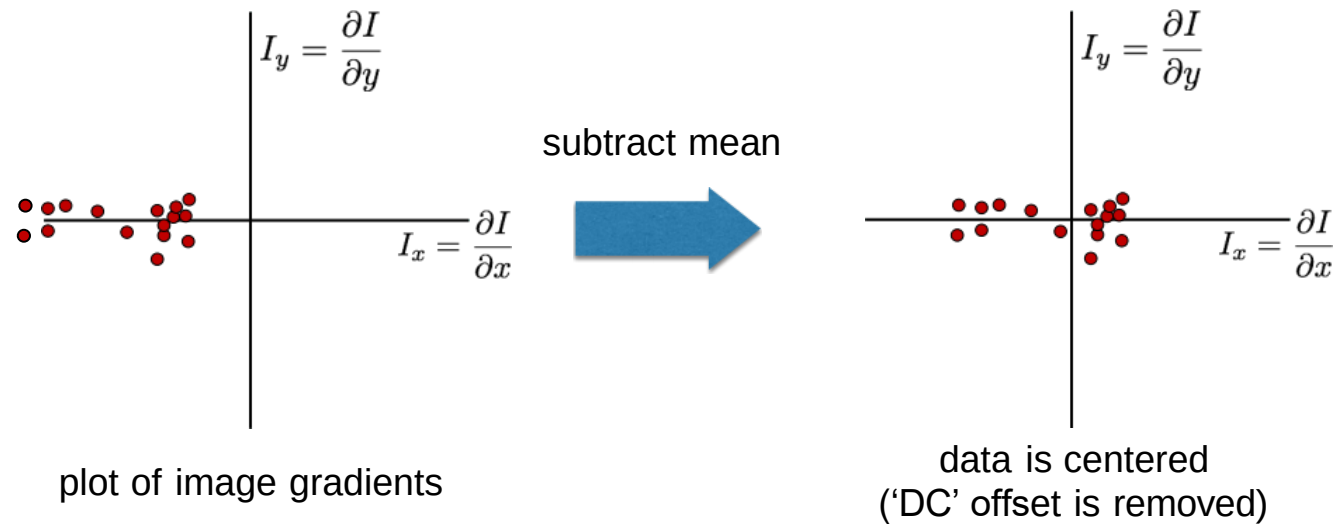
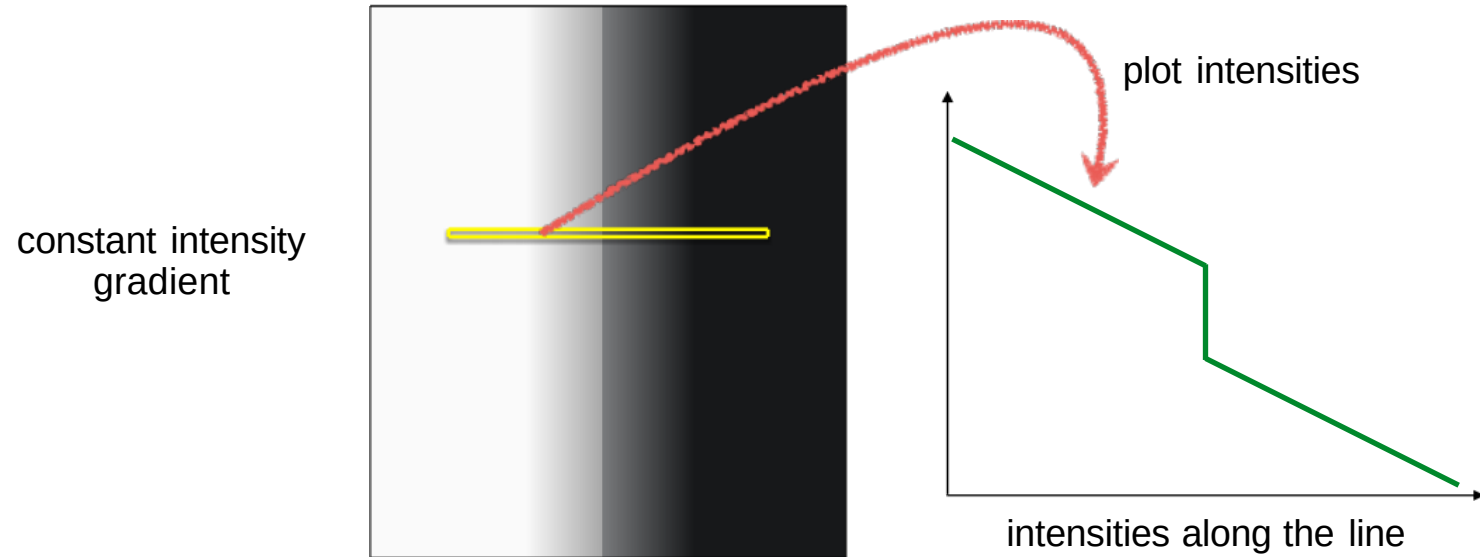


array of y gradients

$$I_y = \frac{\partial I}{\partial y}$$



2. Subtract the mean from each image gradient



3. Compute the covariance matrix

$$\begin{bmatrix} \sum_{p \in P} I_x I_x & \sum_{p \in P} I_x I_y \\ \sum_{p \in P} I_y I_x & \sum_{p \in P} I_y I_y \end{bmatrix}$$

$$\sum_{p \in P} I_x I_y = \text{sum} \left(\begin{array}{c} I_x = \frac{\partial I}{\partial x} \\ \begin{array}{|c|c|c|c|c|c|} \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \end{array} \\ \text{array of x gradients} \end{array} * \begin{array}{c} I_y = \frac{\partial I}{\partial y} \\ \begin{array}{|c|c|c|c|c|c|} \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \end{array} \\ \text{array of y gradients} \end{array} \right)$$

Where does this covariance matrix come from?

Error function

Change of intensity for the shift $[u,v]$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

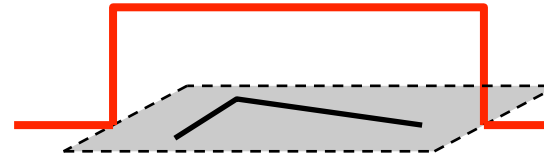
Error
function

Window
function

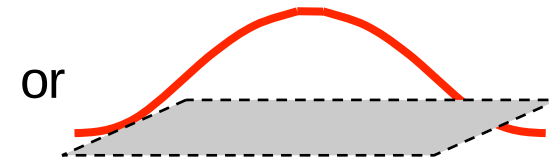
Shifted
intensity

Intensity

Window function $w(x,y) =$



1 in window, 0 outside



Gaussian

Error function approximation

Change of intensity for the shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

First-order Taylor expansion of $I(x,y)$ about $(0,0)$
(bilinear approximation for small shifts)

Bilinear approximation

For small shifts $[u, v]$ we have a 'bilinear approximation':

Change in
appearance for a
shift $[u, v]$

$$E(u, v) \cong [u, v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

where M is a 2×2 matrix computed from image derivatives:

'second moment' matrix
'structure tensor'

$$M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

By computing the gradient covariance matrix...

$$\begin{bmatrix} \sum_{p \in P} I_x I_x & \sum_{p \in P} I_x I_y \\ \sum_{p \in P} I_y I_x & \sum_{p \in P} I_y I_y \end{bmatrix}$$

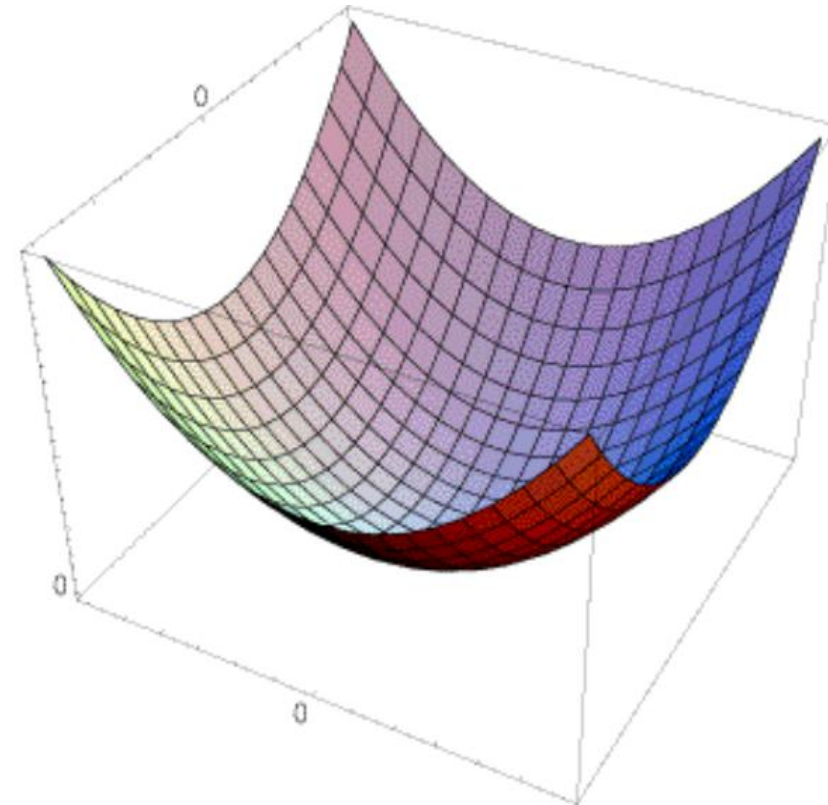
we are fitting a quadratic to the gradients over a small image region

Visualization of a quadratic

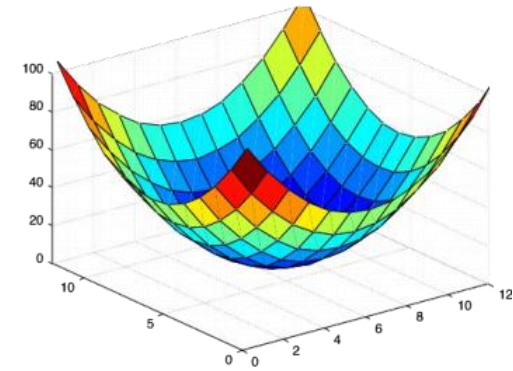
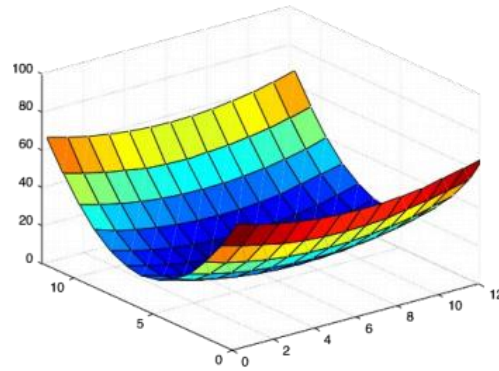
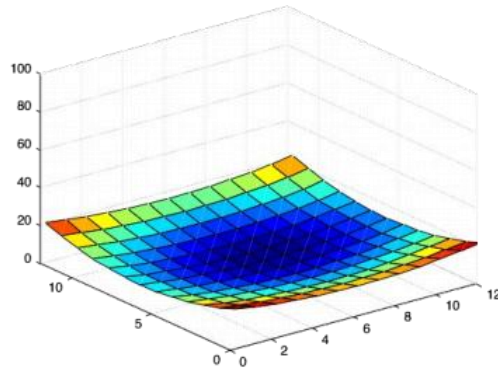
The surface $E(u,v)$ is locally approximated by a quadratic form

$$E(u,v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

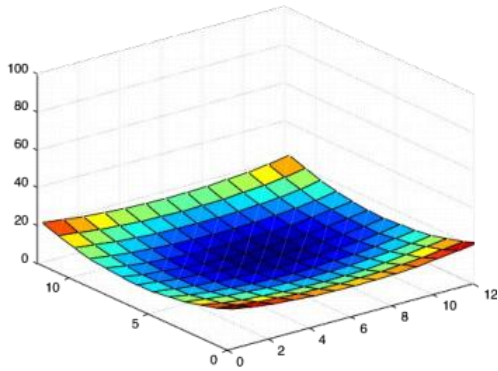


Which error surface indicates a good image feature?

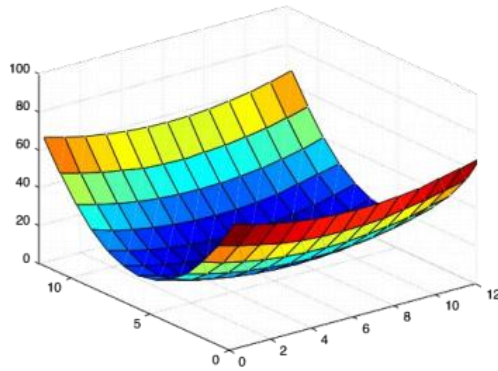


What kind of image patch do these surfaces represent?

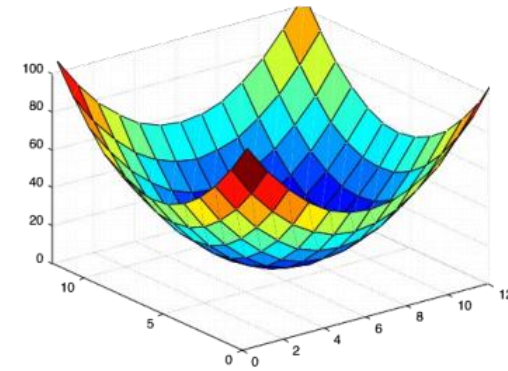
Which error surface indicates a good image feature?



flat



edge
'line'



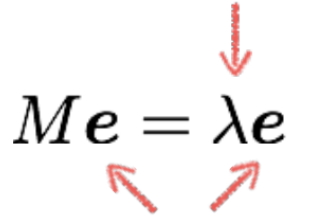
corner
'dot'

4. Compute eigenvalues and eigenvectors

eigenvalue

$$M\mathbf{e} = \lambda\mathbf{e}$$

eigenvector



$$(M - \lambda I)\mathbf{e} = 0$$

1. Compute the determinant of
(returns a polynomial)

$$M - \lambda I$$

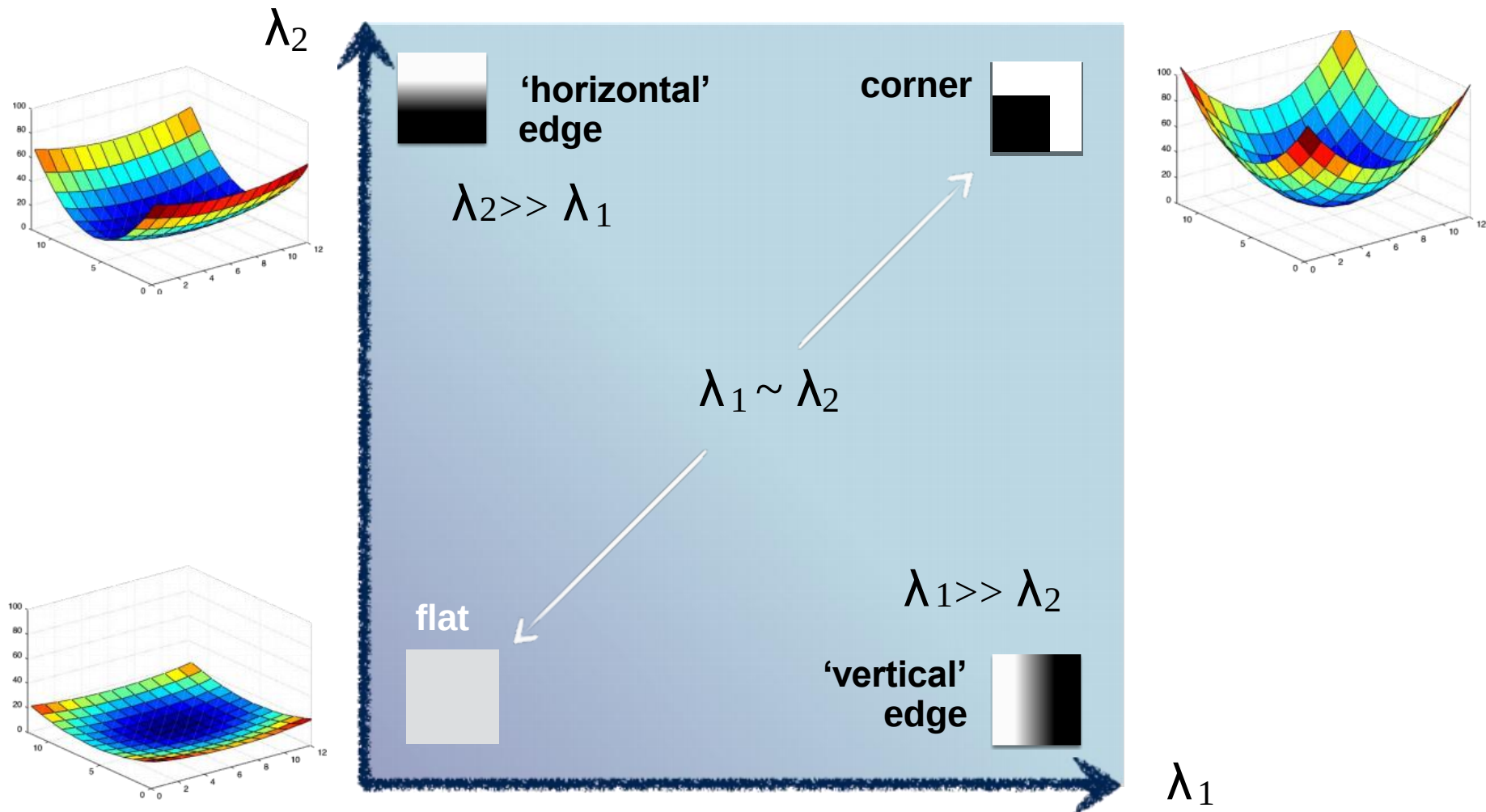
2. Find the roots of polynomial
(returns eigenvalues)

$$\det(M - \lambda I) = 0$$

3. For each eigenvalue, solve
(returns eigenvectors)

$$(M - \lambda I)\mathbf{e} = 0$$

interpreting eigenvalues



Harris & Stephens (1988)

$$R = \det(M) - \kappa \text{trace}^2(M)$$

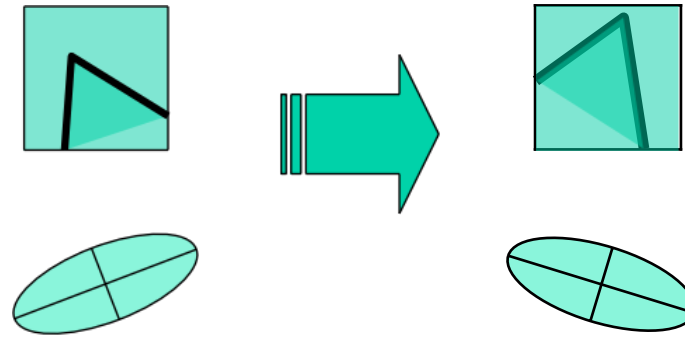
Kanade & Tomasi (1994)

$$R = \min(\lambda_1, \lambda_2)$$

Nobel (1998)

$$R = \frac{\det(M)}{\text{trace}(M) + \epsilon}$$

Harris corner response is invariant to rotation



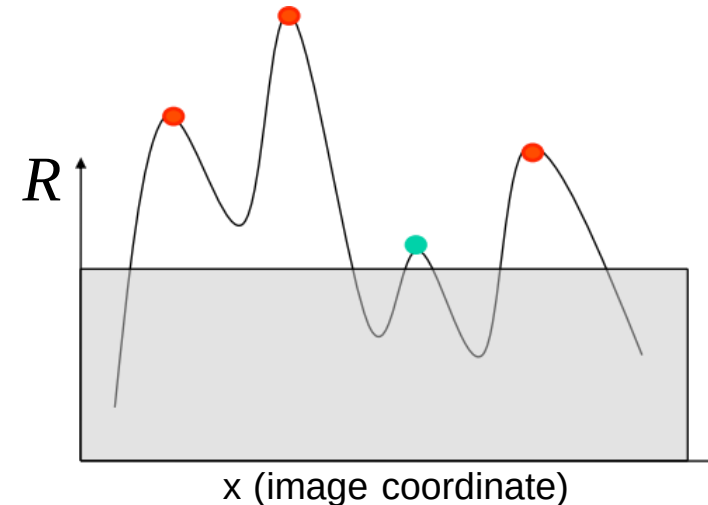
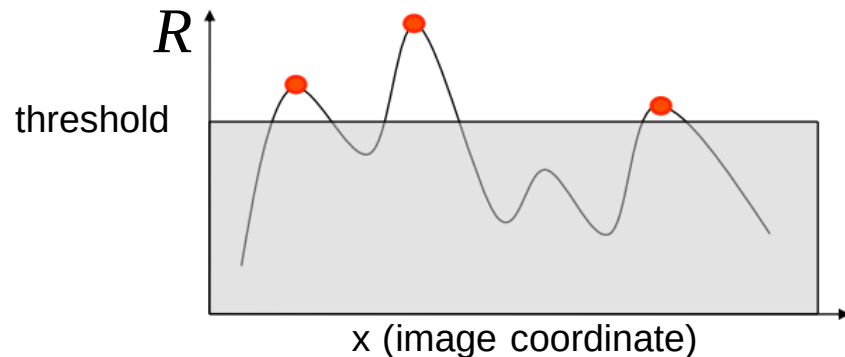
Ellipse rotates but its shape
(**eigenvalues**) remains the same

Corner response R is invariant to image rotation

Harris corner response is invariant to intensity changes

Partial invariance to *affine intensity* change

- Only derivatives are used => invariance to intensity shift $I \rightarrow I + b$
- Intensity scale: $I \rightarrow a I$



The Harris corner detector is not
invariant to scale

edge!



corner!



Multi-scale detection

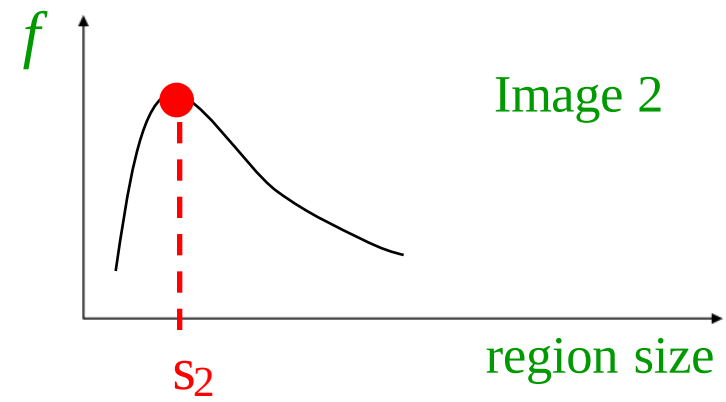
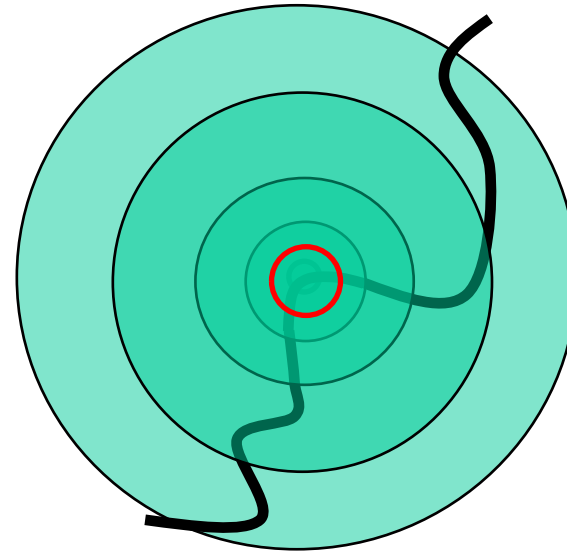
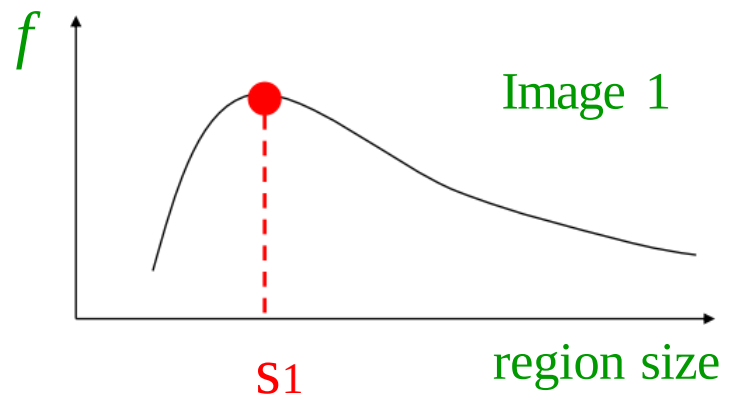
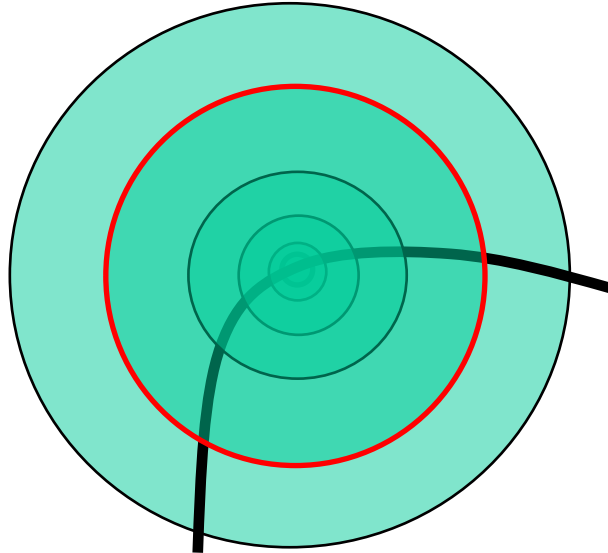
How can we make a feature detector scale-invariant?

How can we automatically select the scale?



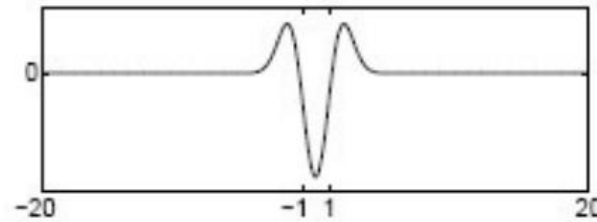
Intuitively ...

Find local maxima in both **position** and **scale**

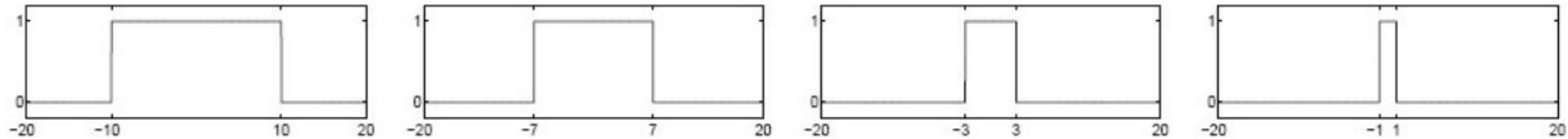


Formally ...

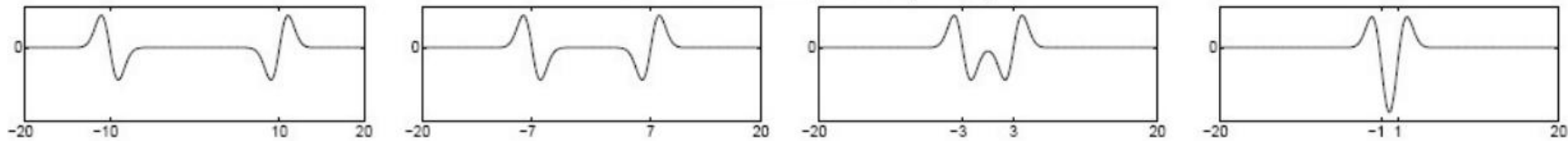
Laplacian filter



Original signal

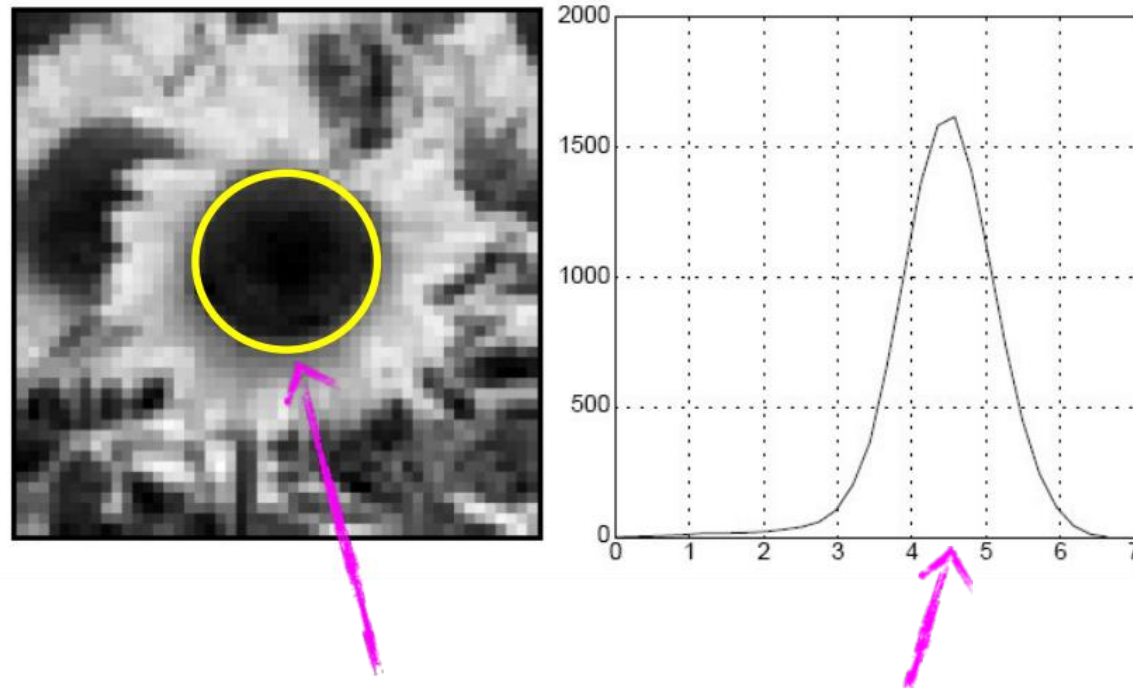


Convolved with Laplacian ($\sigma = 1$)



Highest response when the signal has the same **characteristic scale** as the filter

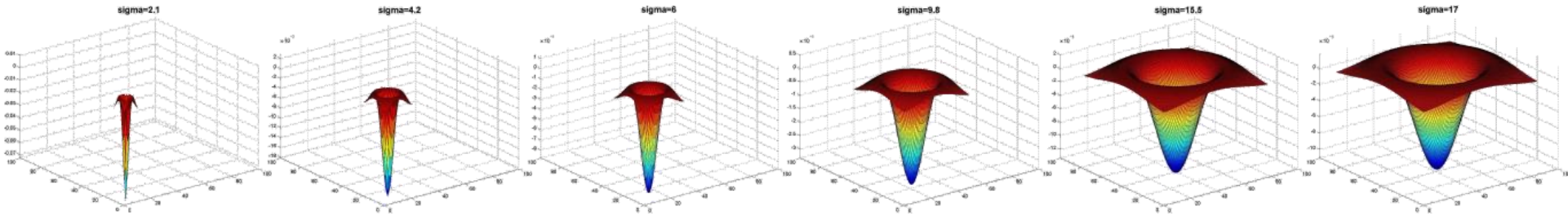
characteristic scale - the scale that
produces peak filter response



characteristic scale

we need to search over characteristic scales

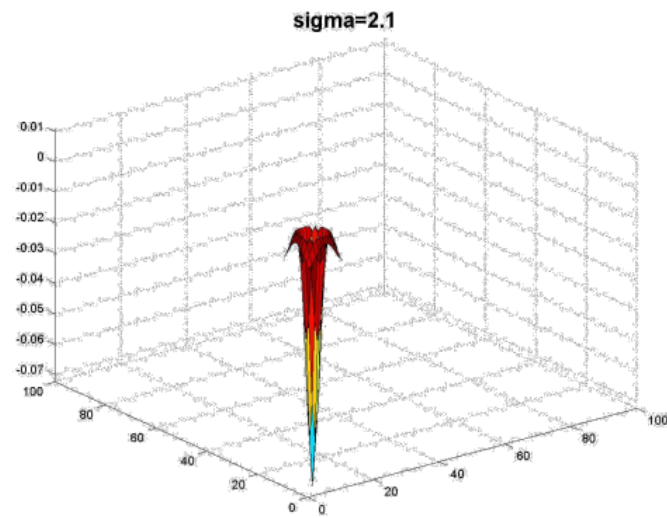
What happens if you apply different Laplacian filters?



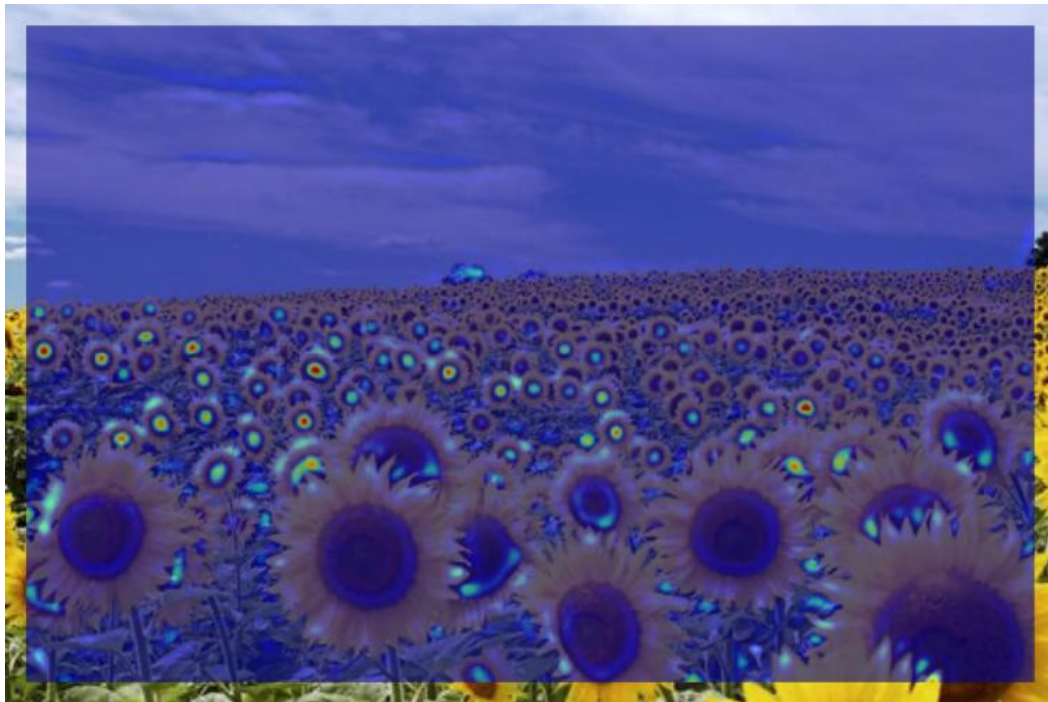
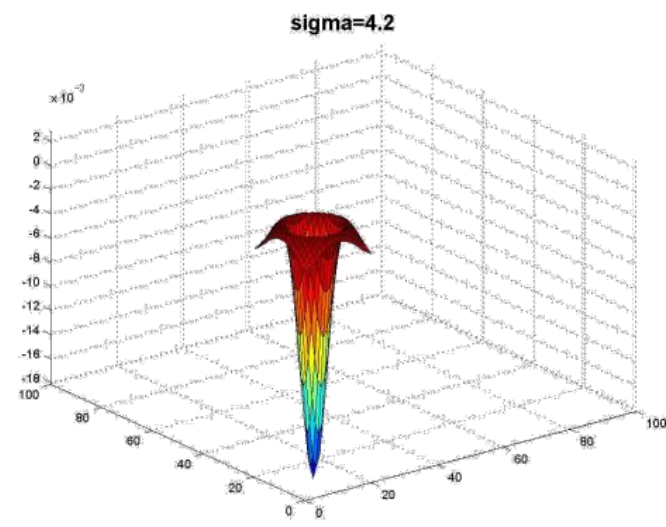
Full size

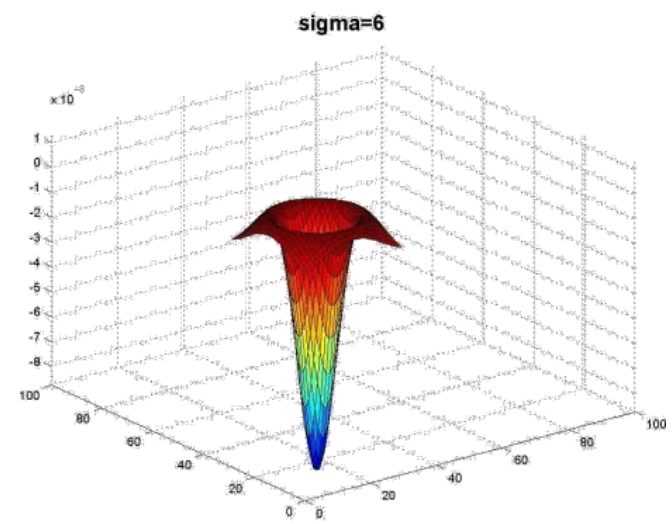
3/4 size

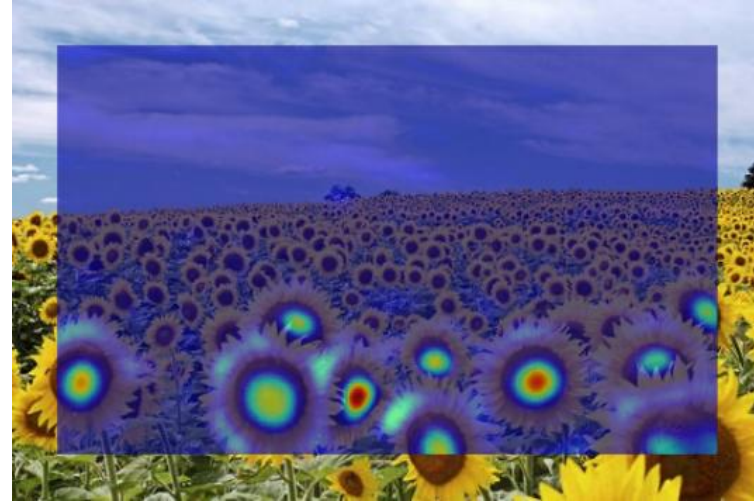
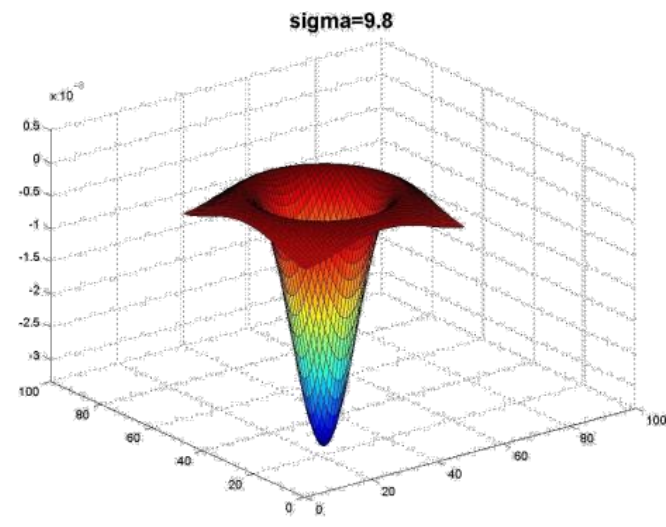


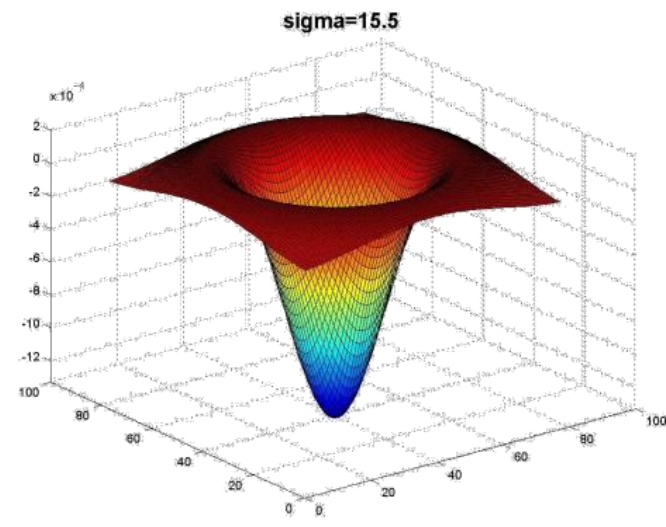


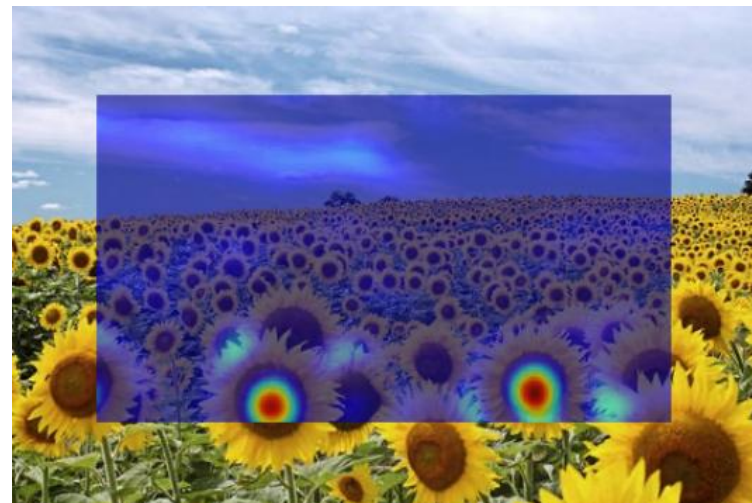
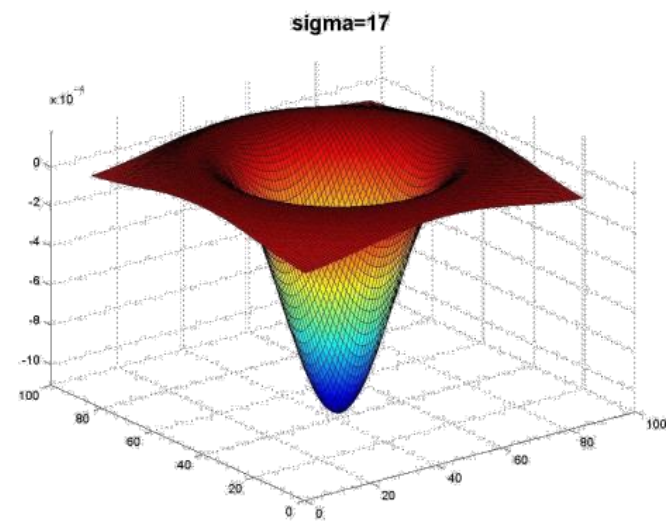
jet color scale
blue: low, red: high











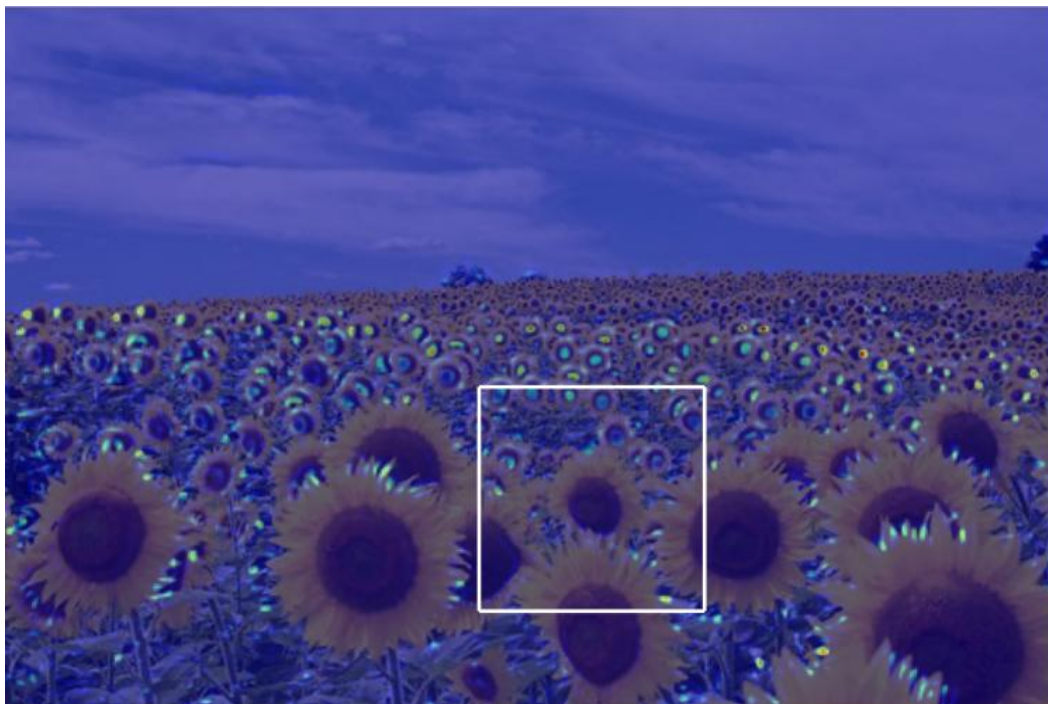
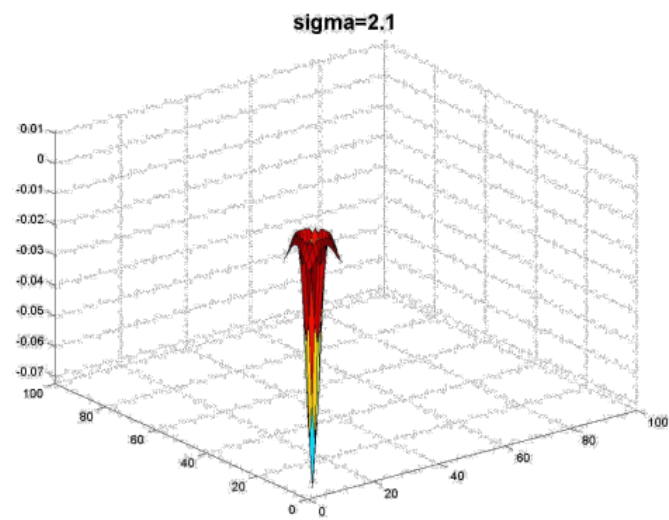
What happened when you applied different Laplacian filters?

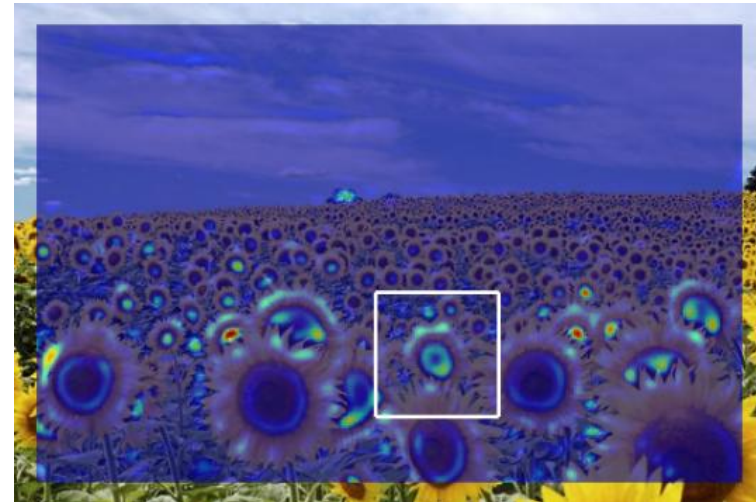
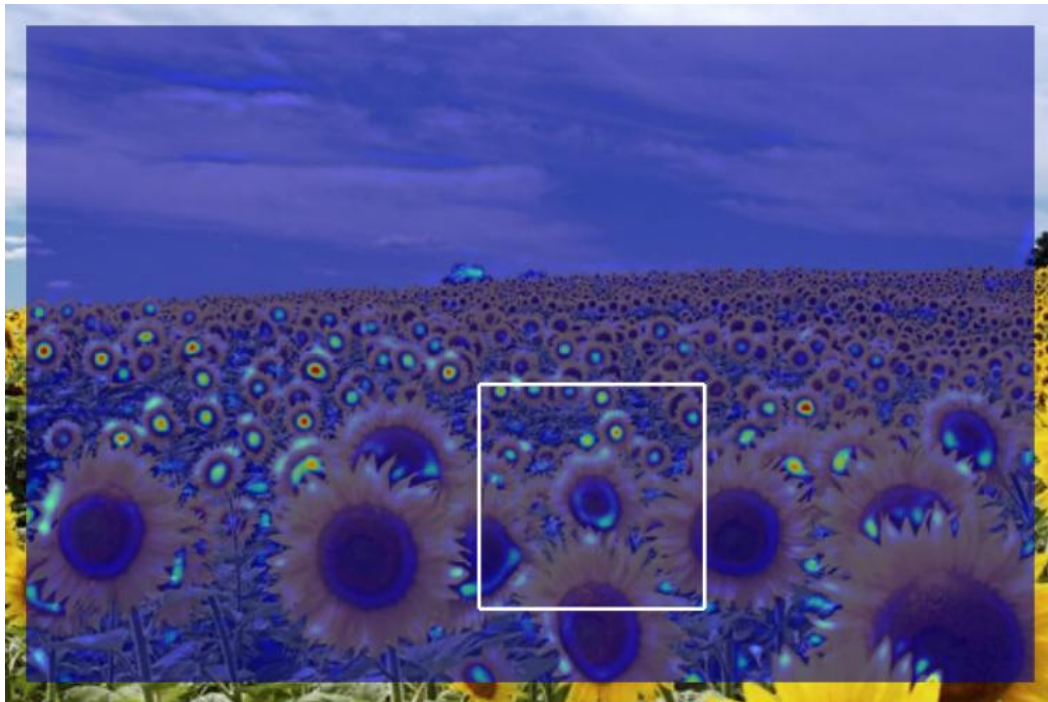
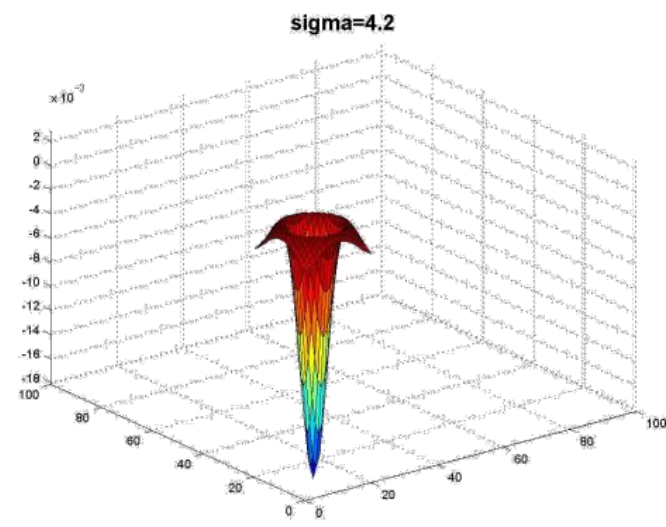
Full size

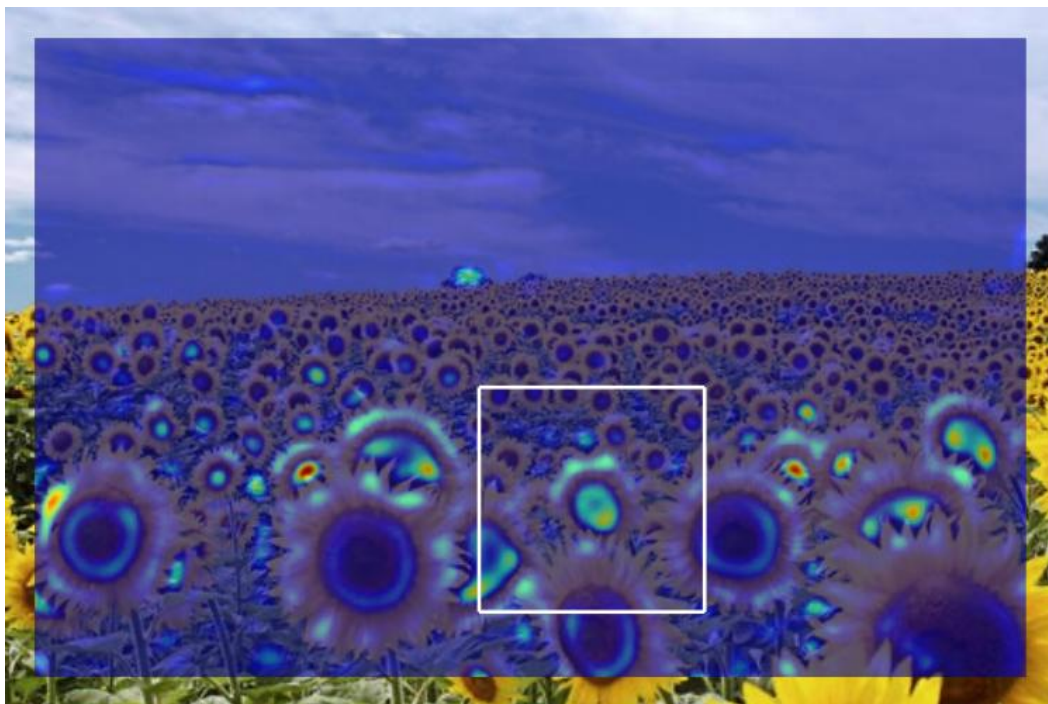
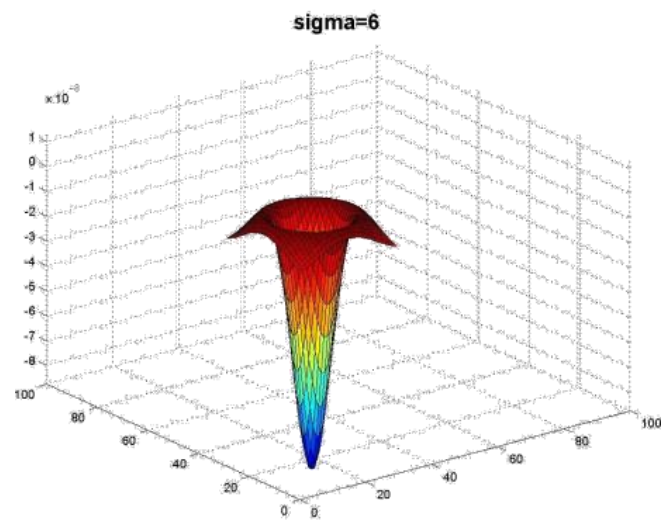


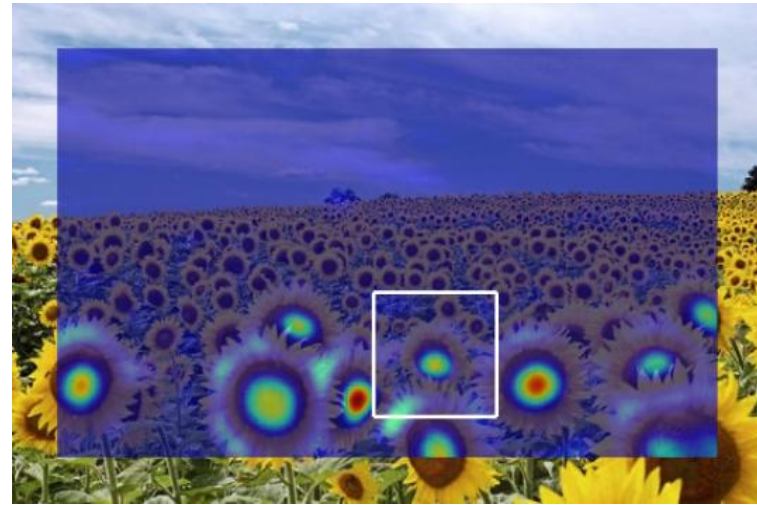
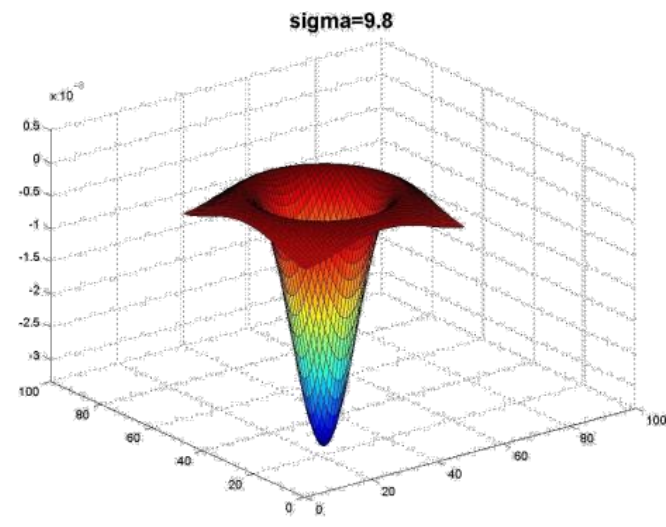
3/4 size

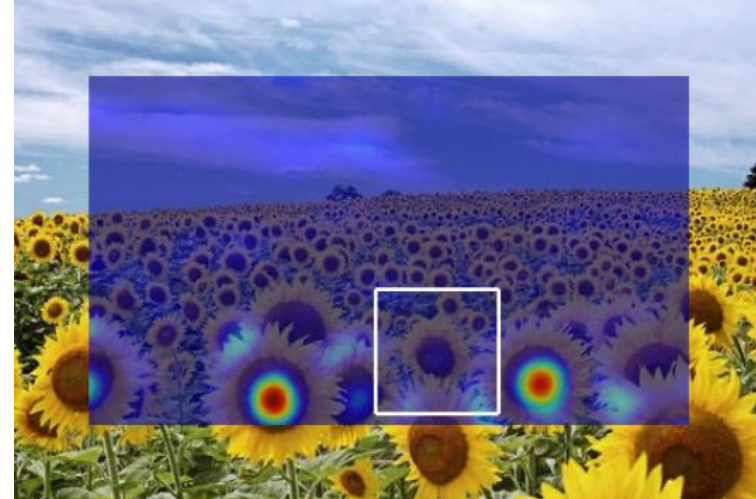
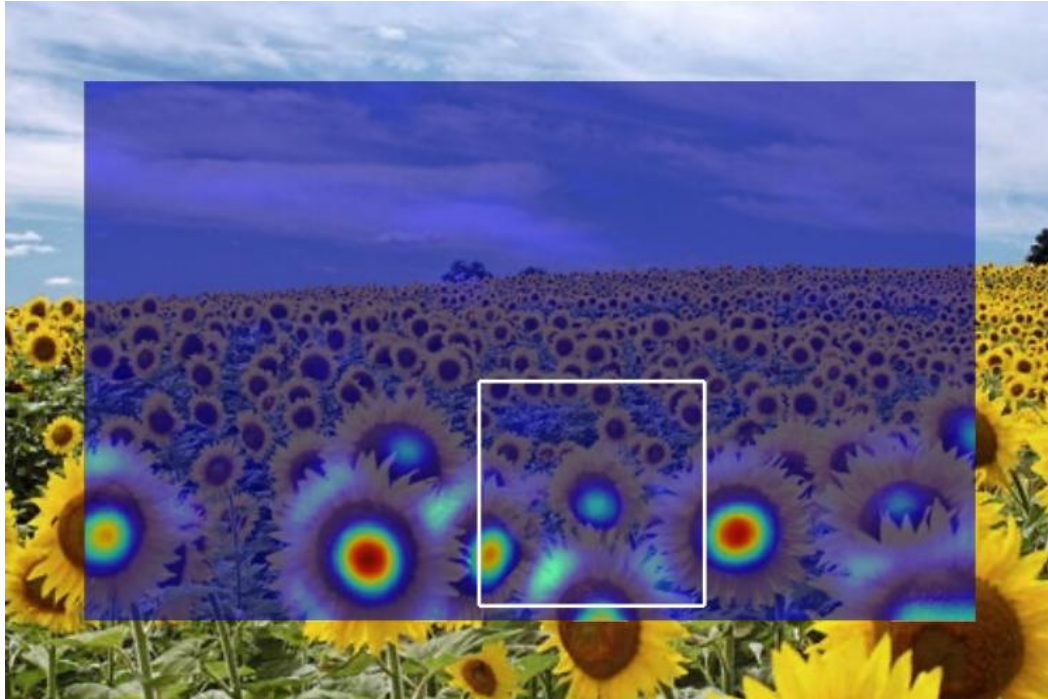
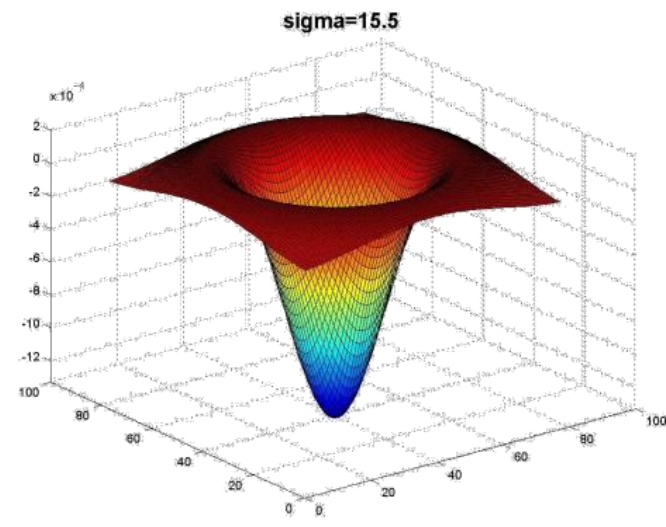


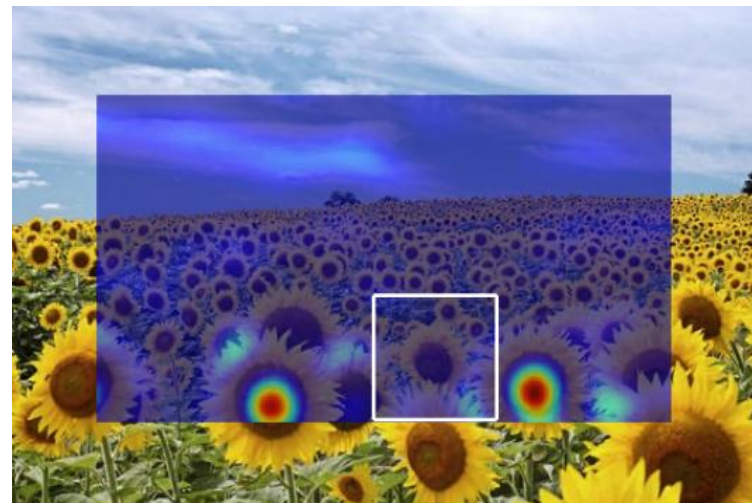
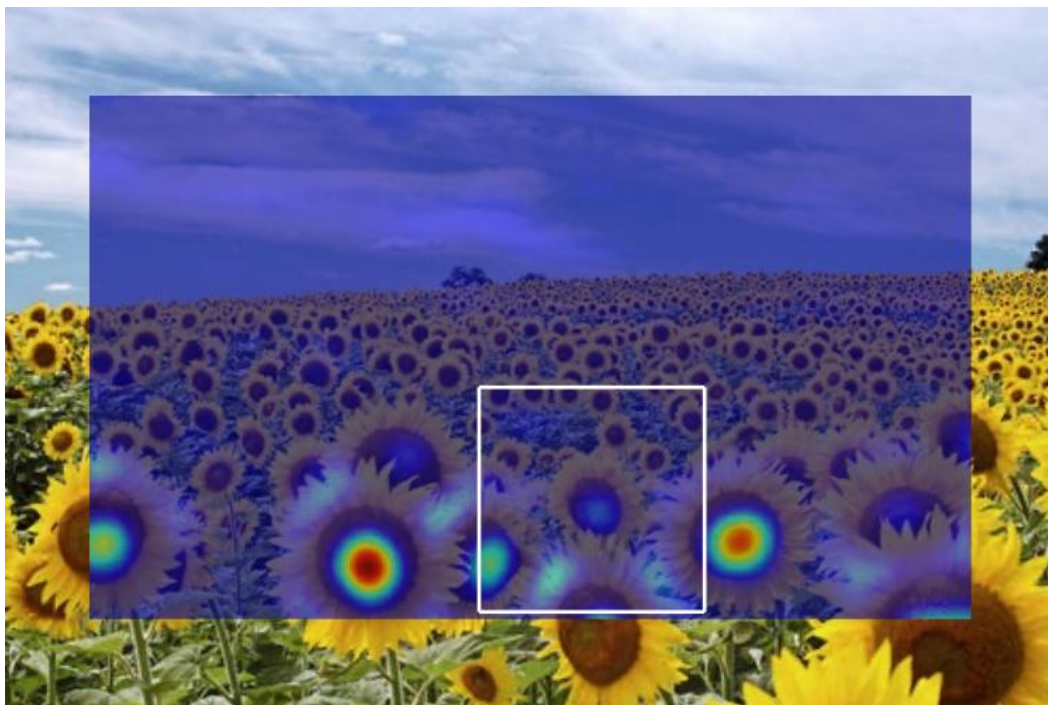
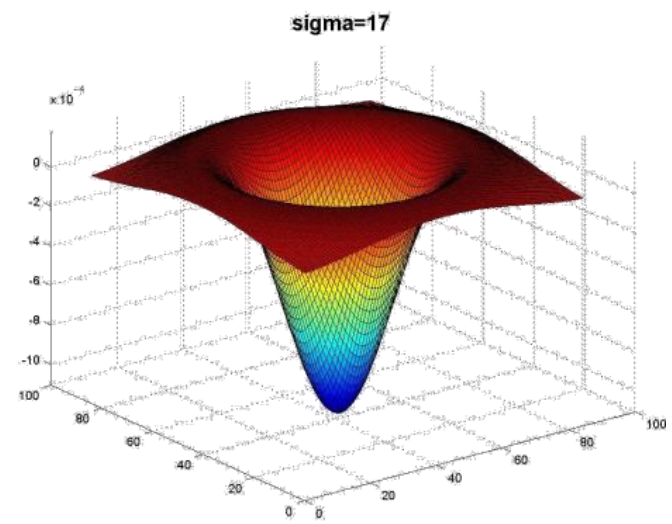




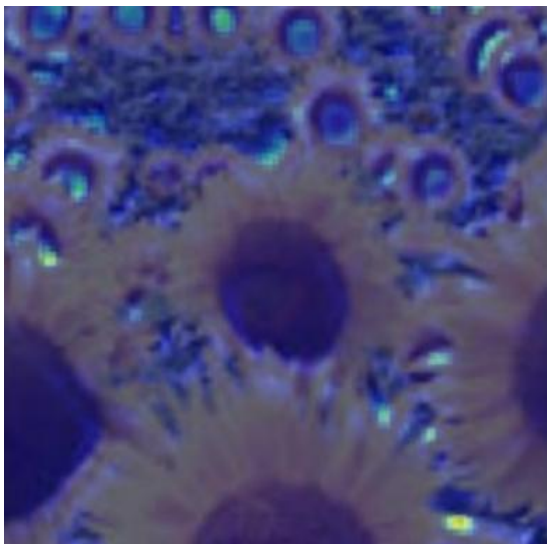




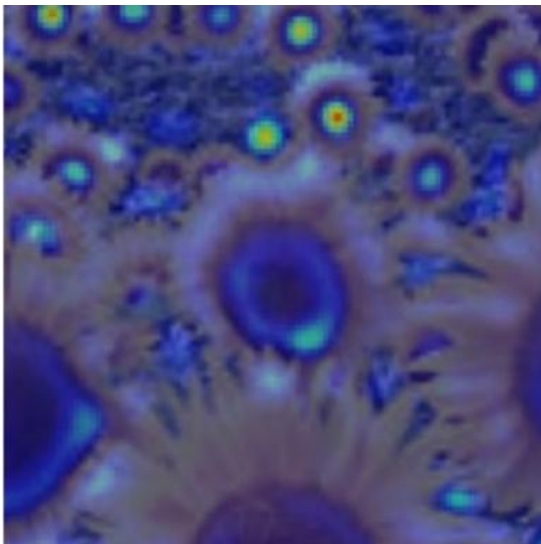




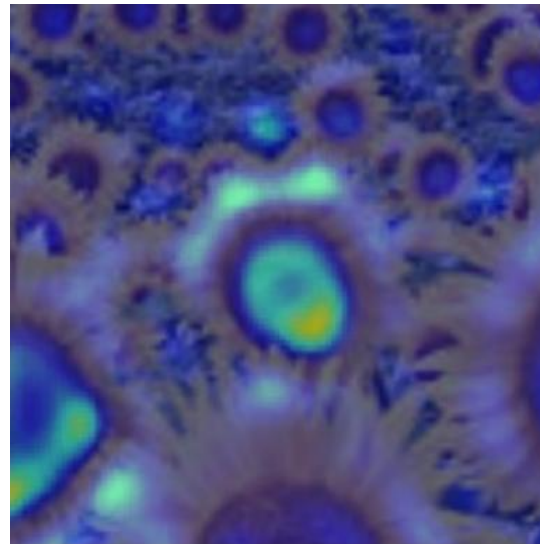
2.1



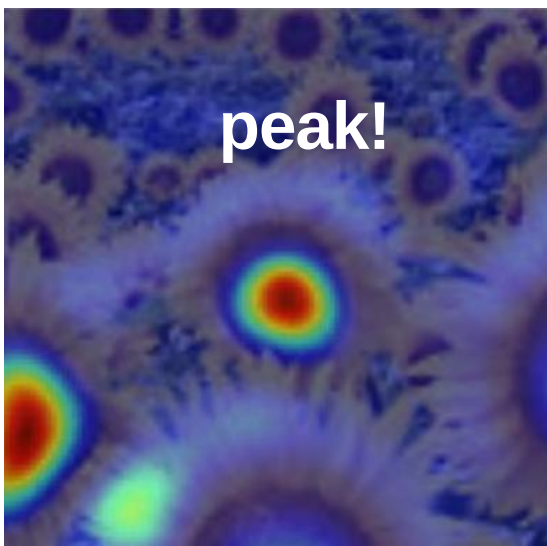
4.2



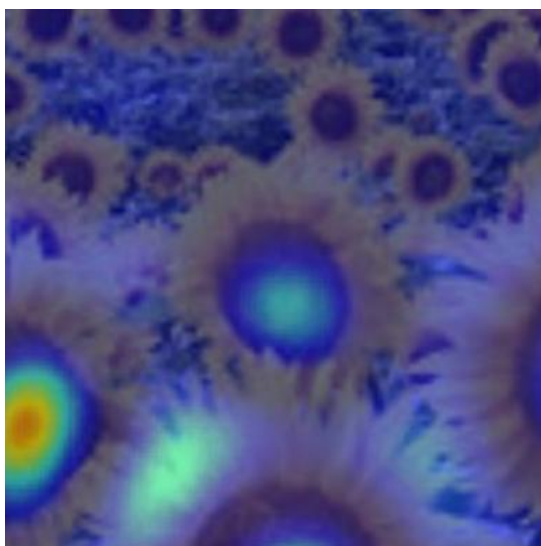
6.0



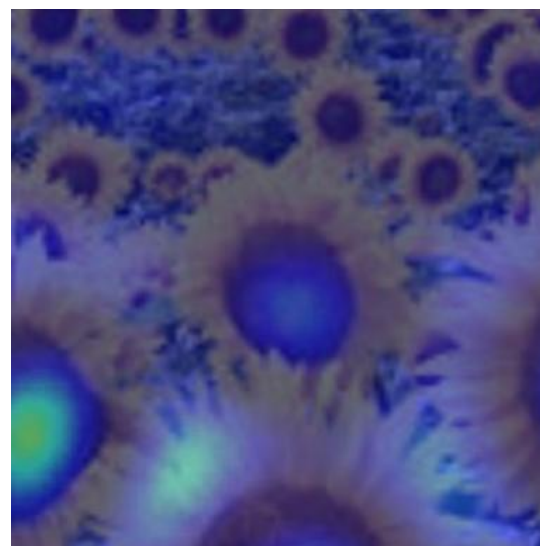
9.8



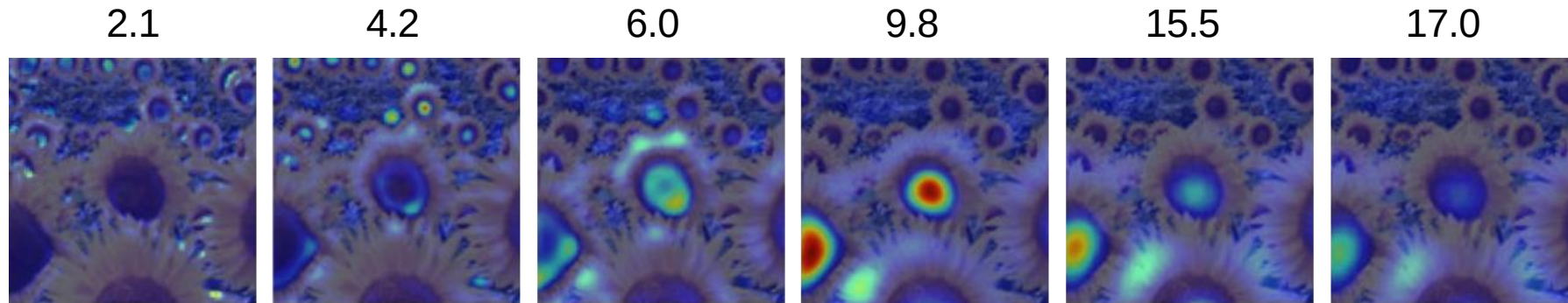
15.5



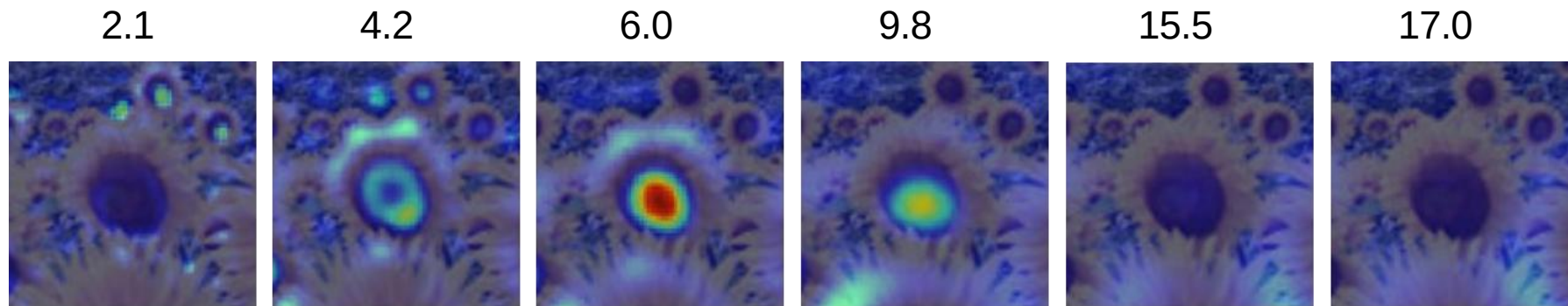
17.0



optimal scale

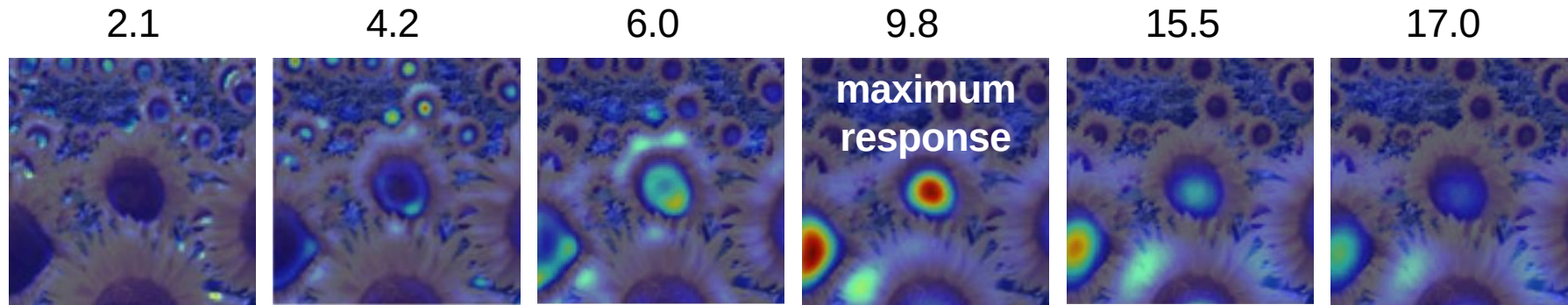


Full size image

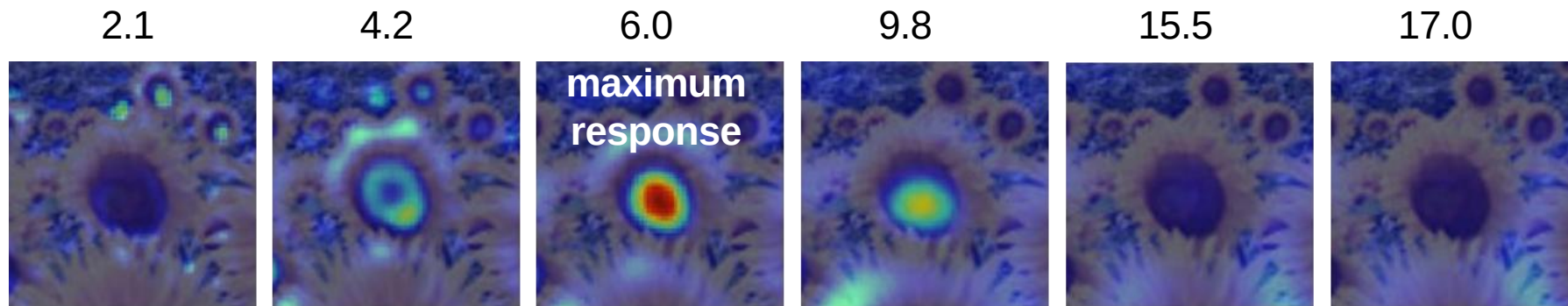


3/4 size image

optimal scale

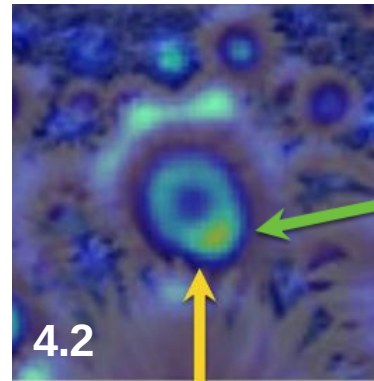


Full size image

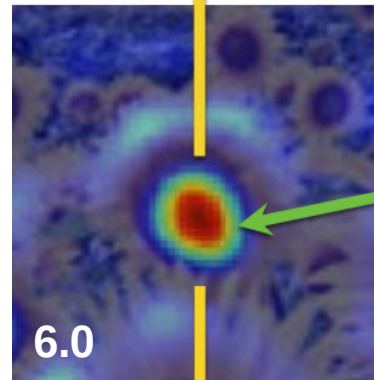


3/4 size image

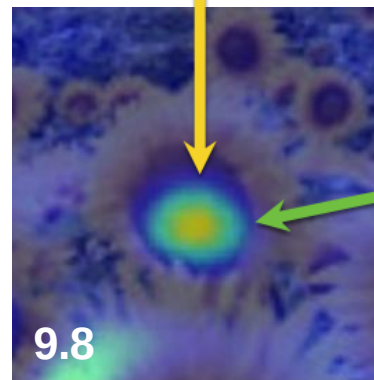
cross-scale maximum



local maximum



local maximum



local maximum

How would you
implement scale
selection?

Implementation

For each level of the Gaussian pyramid

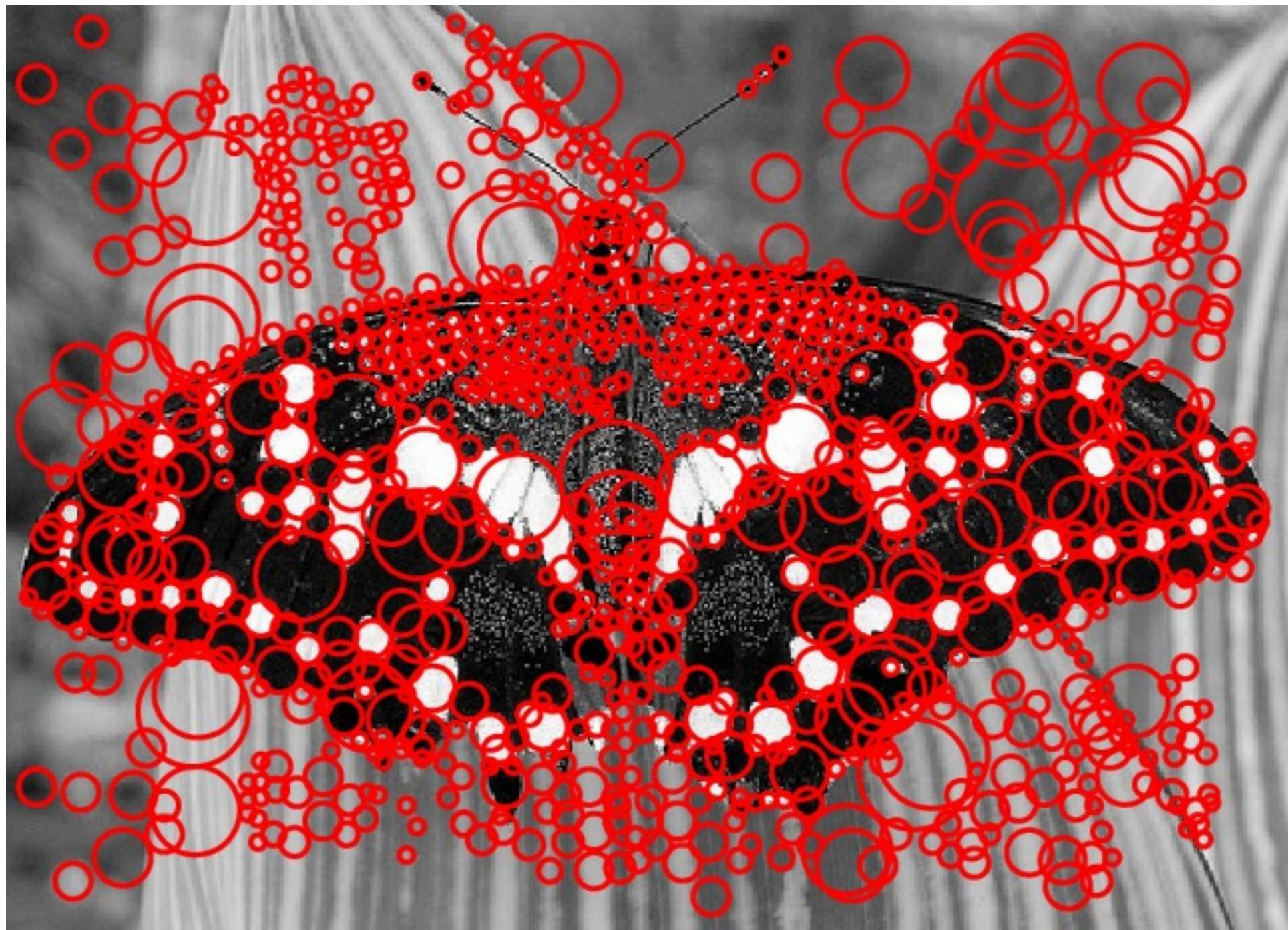
 compute feature response (e.g. Harris, Laplacian)

For each level of the Gaussian pyramid

 if local maximum and cross-scale

save scale and location of feature (x, y, s)





Reminder: image transformations

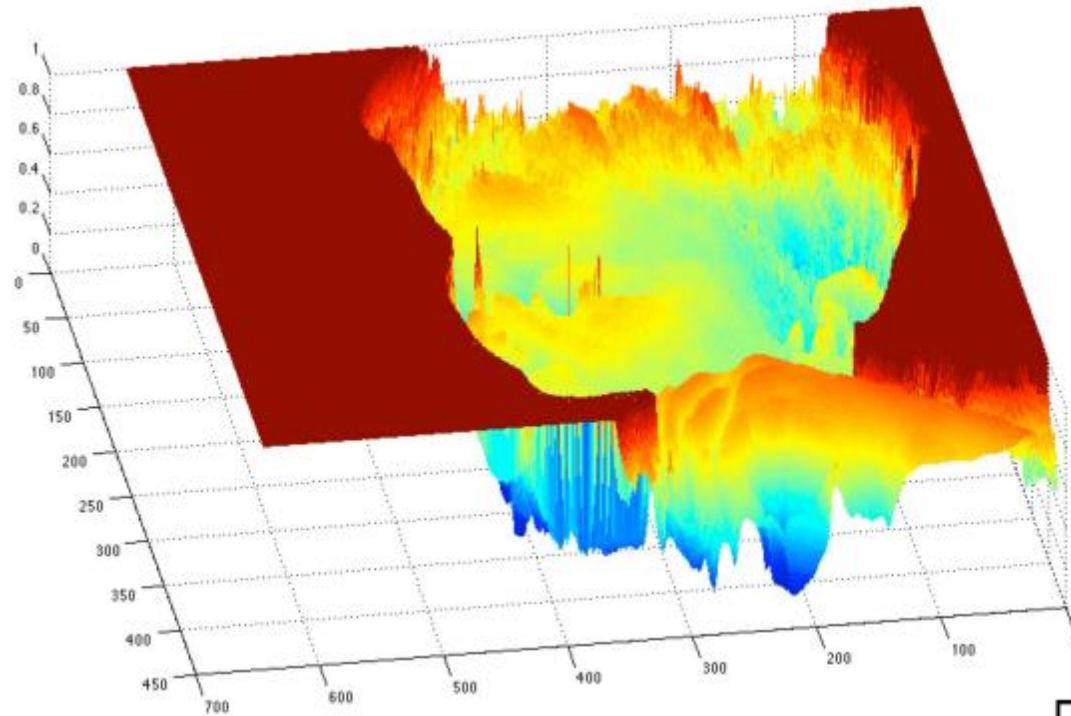
What is an image?

$f(\mathbf{ar})$



grayscale image

What is the range of the image function f ?



domain $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

A (grayscale) image is a 2D function.

What types of image transformations can we do?



Filtering



changes pixel *values*



Warping



changes pixel *locations*

What types of image transformations can we do?

F



Filtering



$$G(\mathbf{x}) = h\{F(\mathbf{x})\}$$

G



changes *range* of image function

F



Warping



$$G(\mathbf{x}) = F(h\{\mathbf{x}\})$$

G



changes *domain* of image function

Warping example: feature matching



Warping example: feature matching



Warping example: feature matching

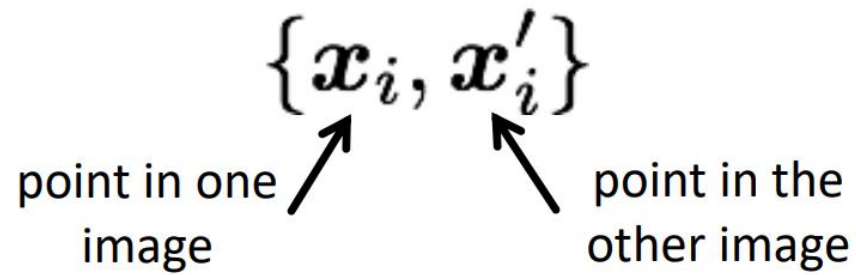


- object recognition
- 3D reconstruction
- augmented reality
- image stitching

How do you compute the transformation?

Warping example: feature matching

Given a set of matched feature points:



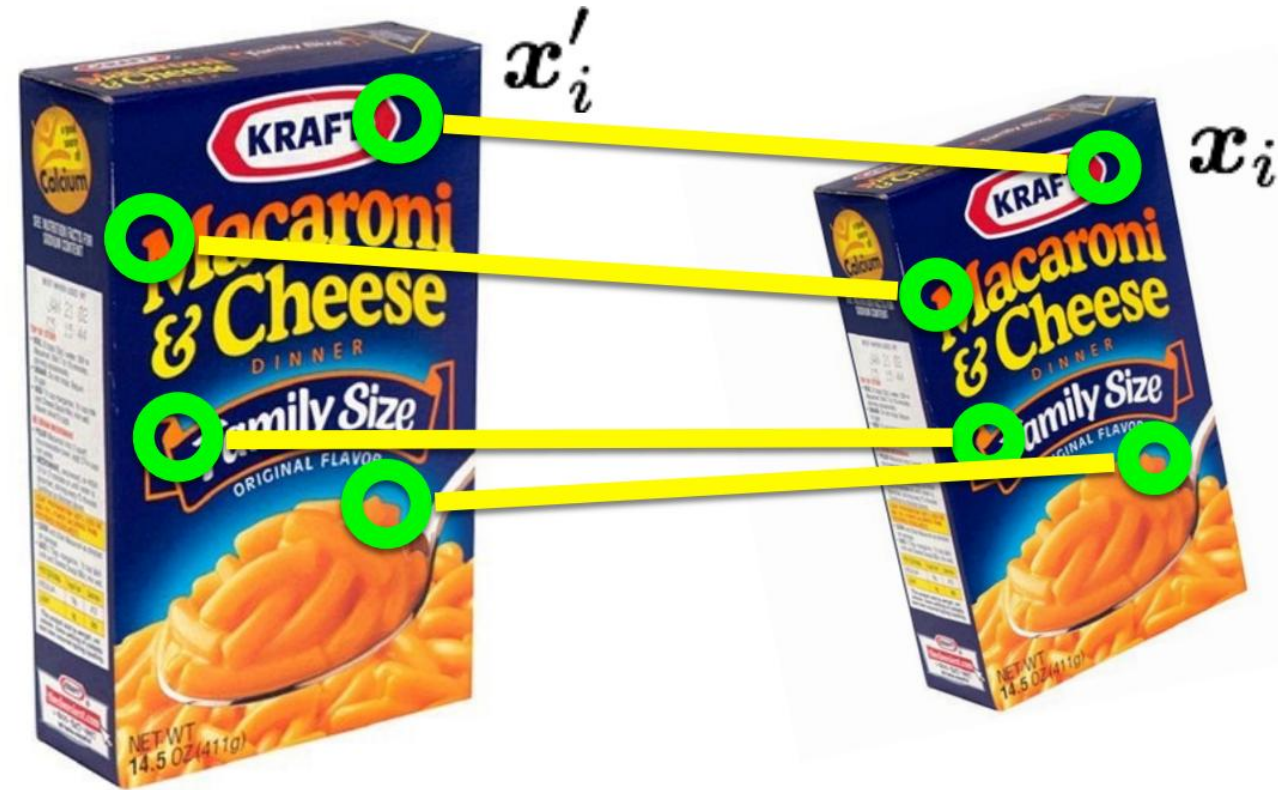
and a transformation:

$$x' = f(x; p)$$

transformation function parameters

find the best estimate of the parameters

p



What kind of transformation functions f are there?

2D transformations

2D transformations



translation



rotation



aspect



affine



perspective

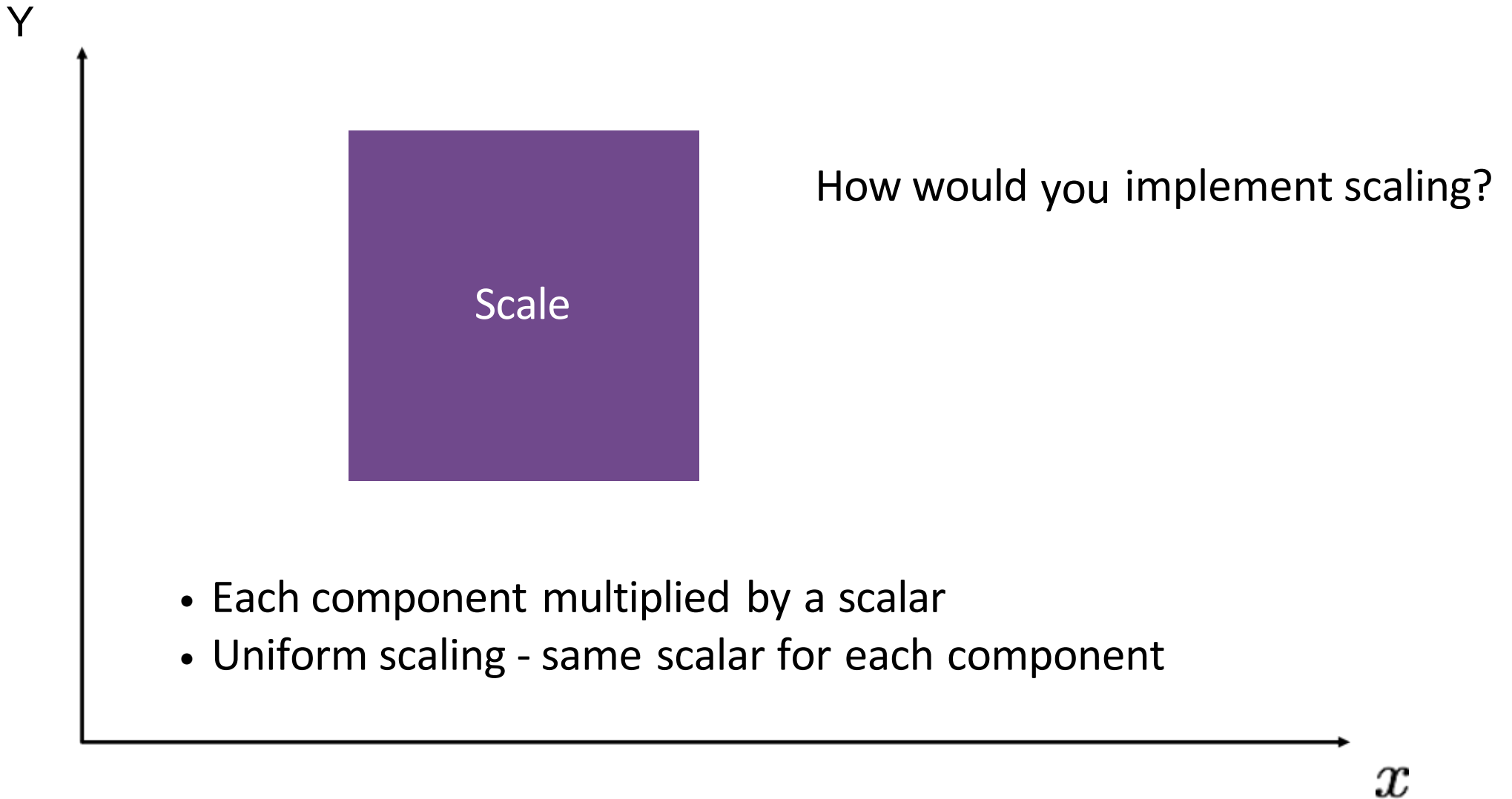


cylindrical

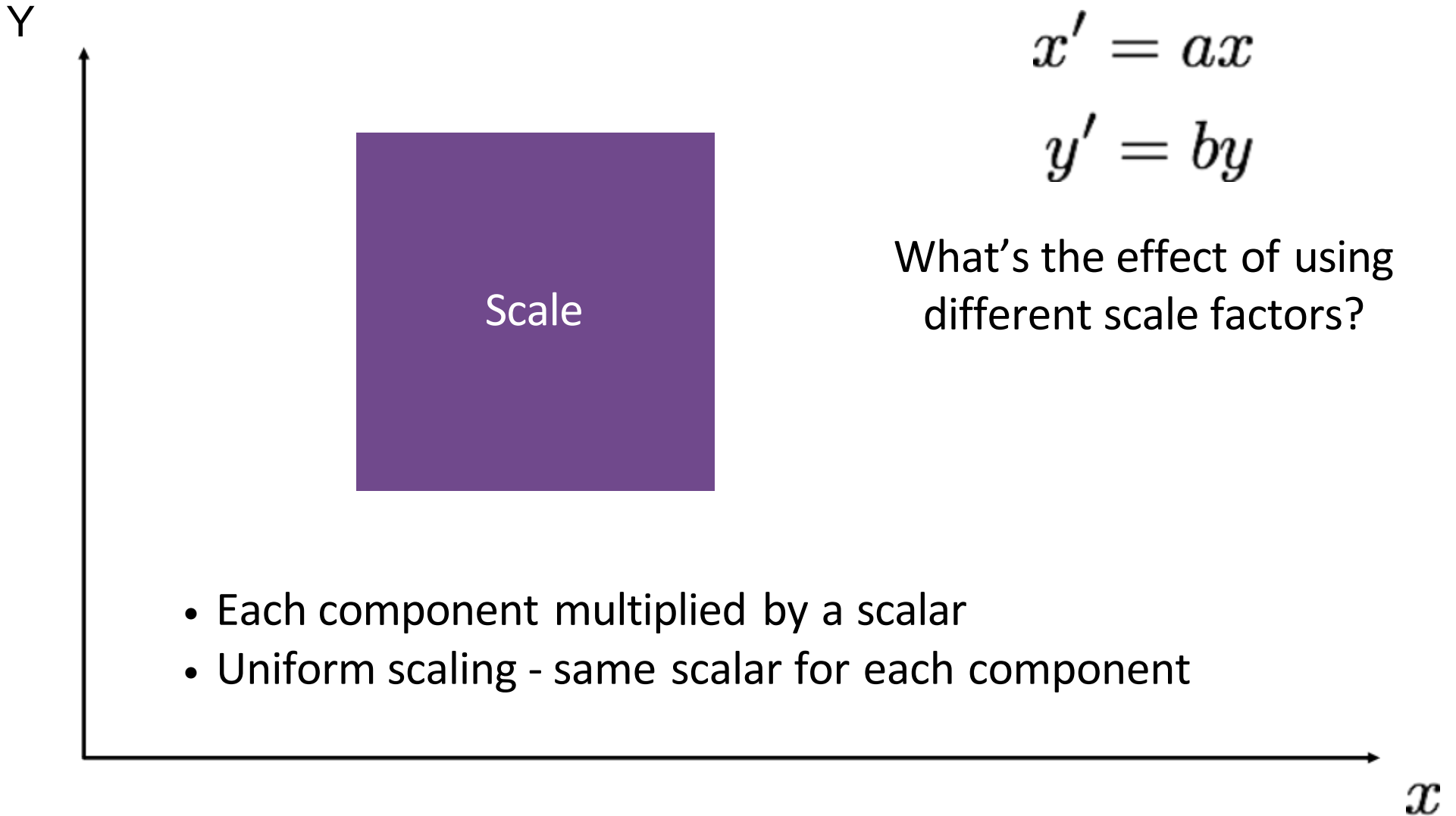
2D planar transformations



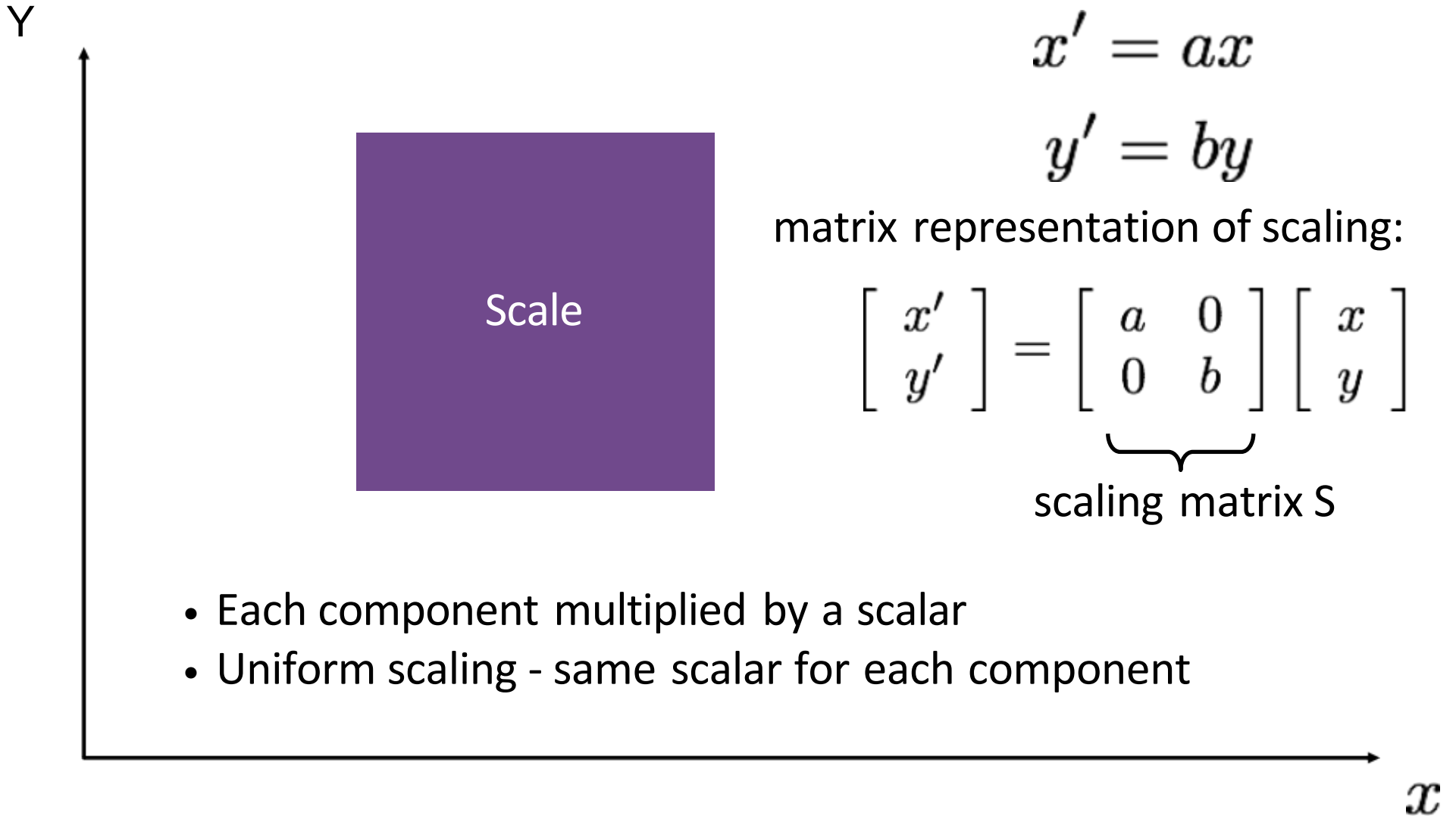
2D planar transformations



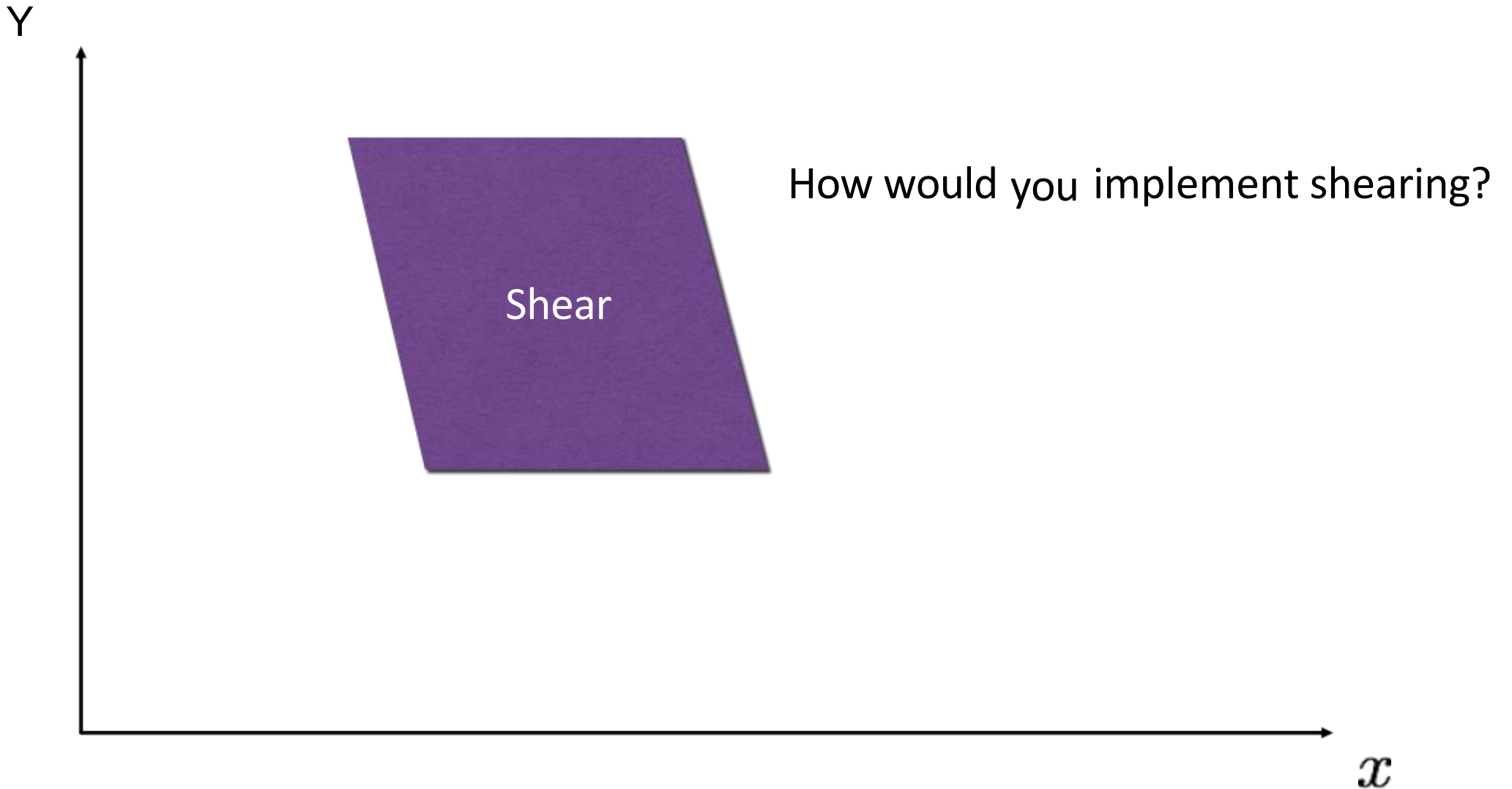
2D planar transformations



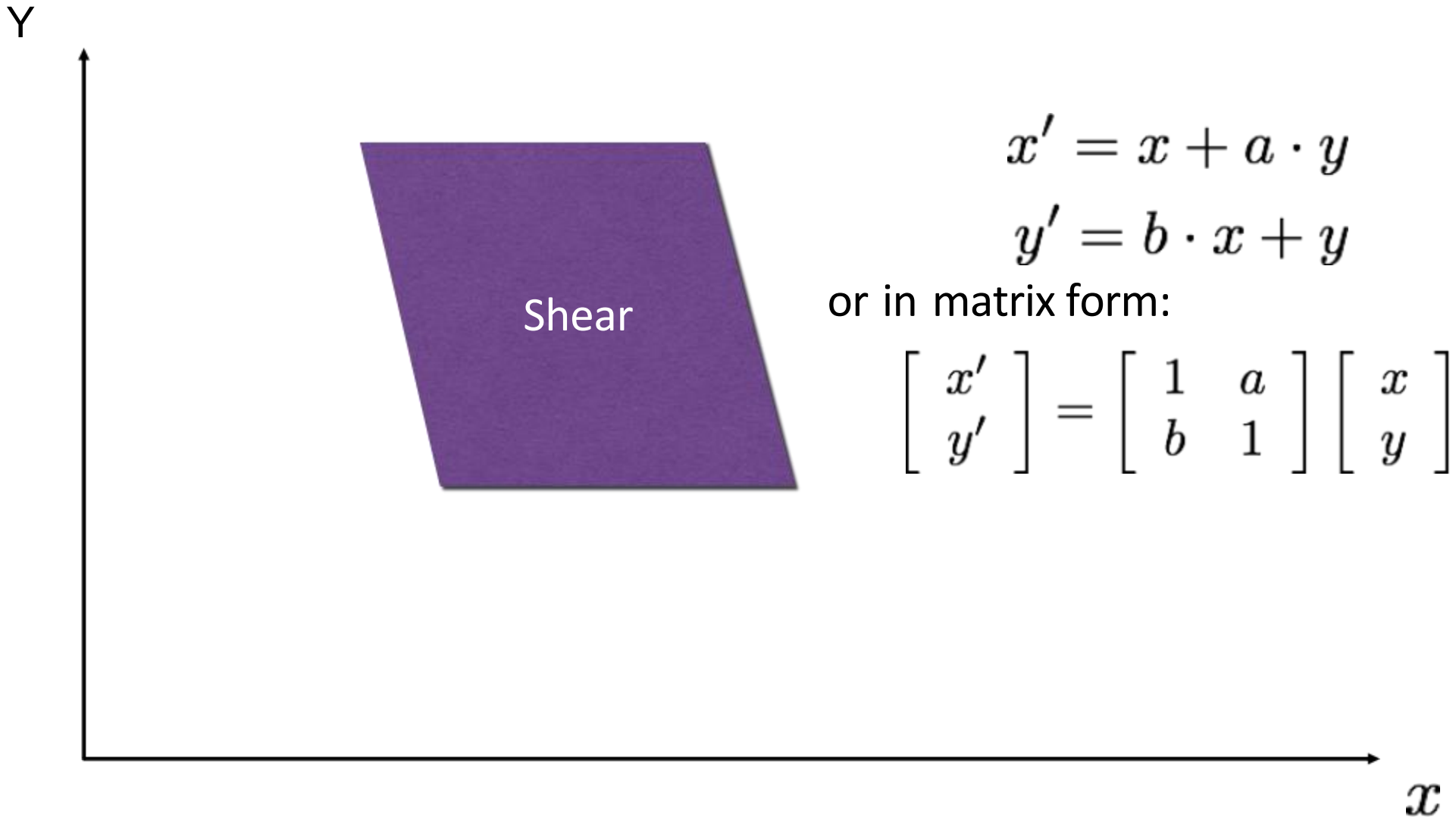
2D planar transformations



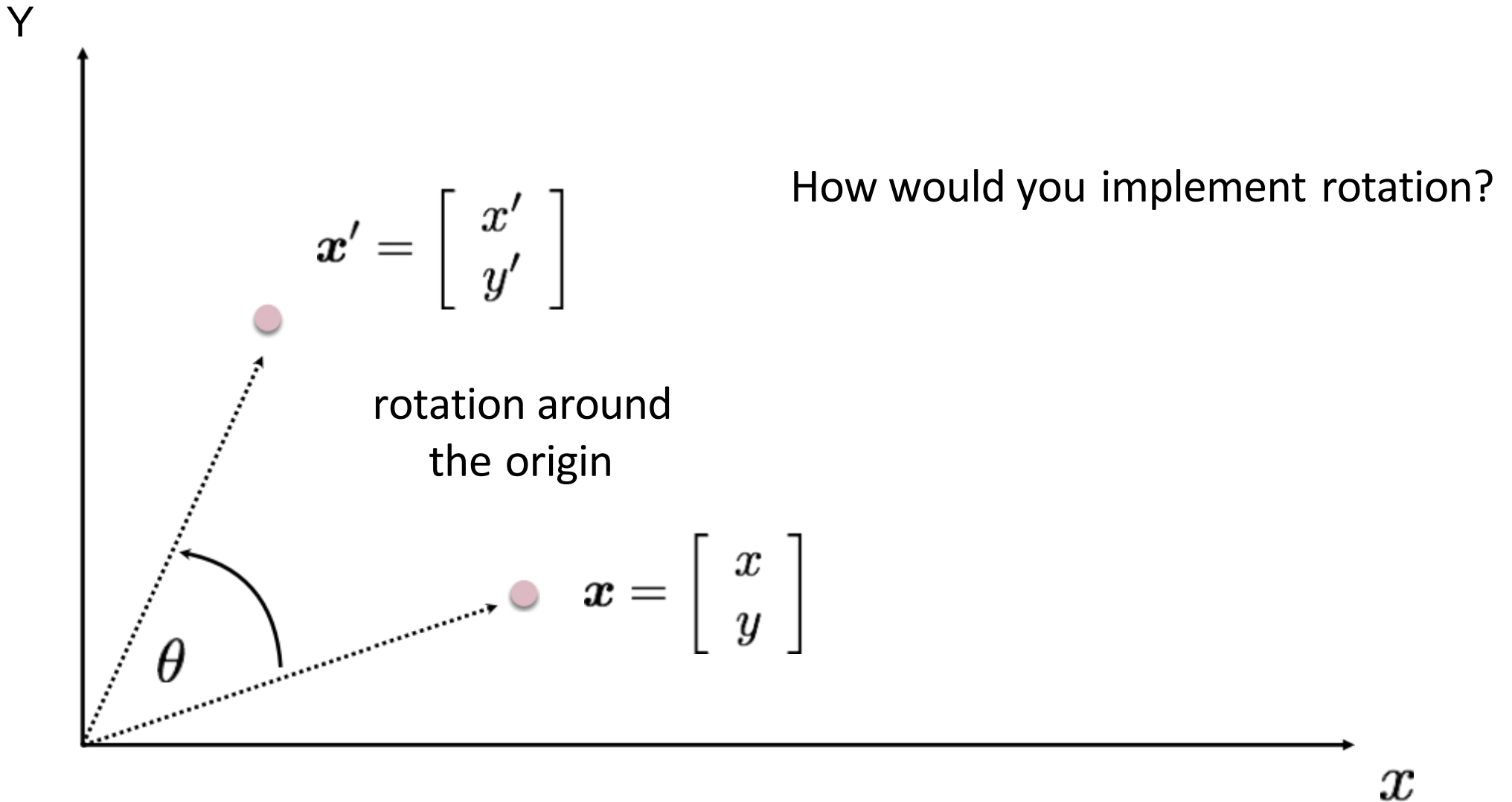
2D planar transformations



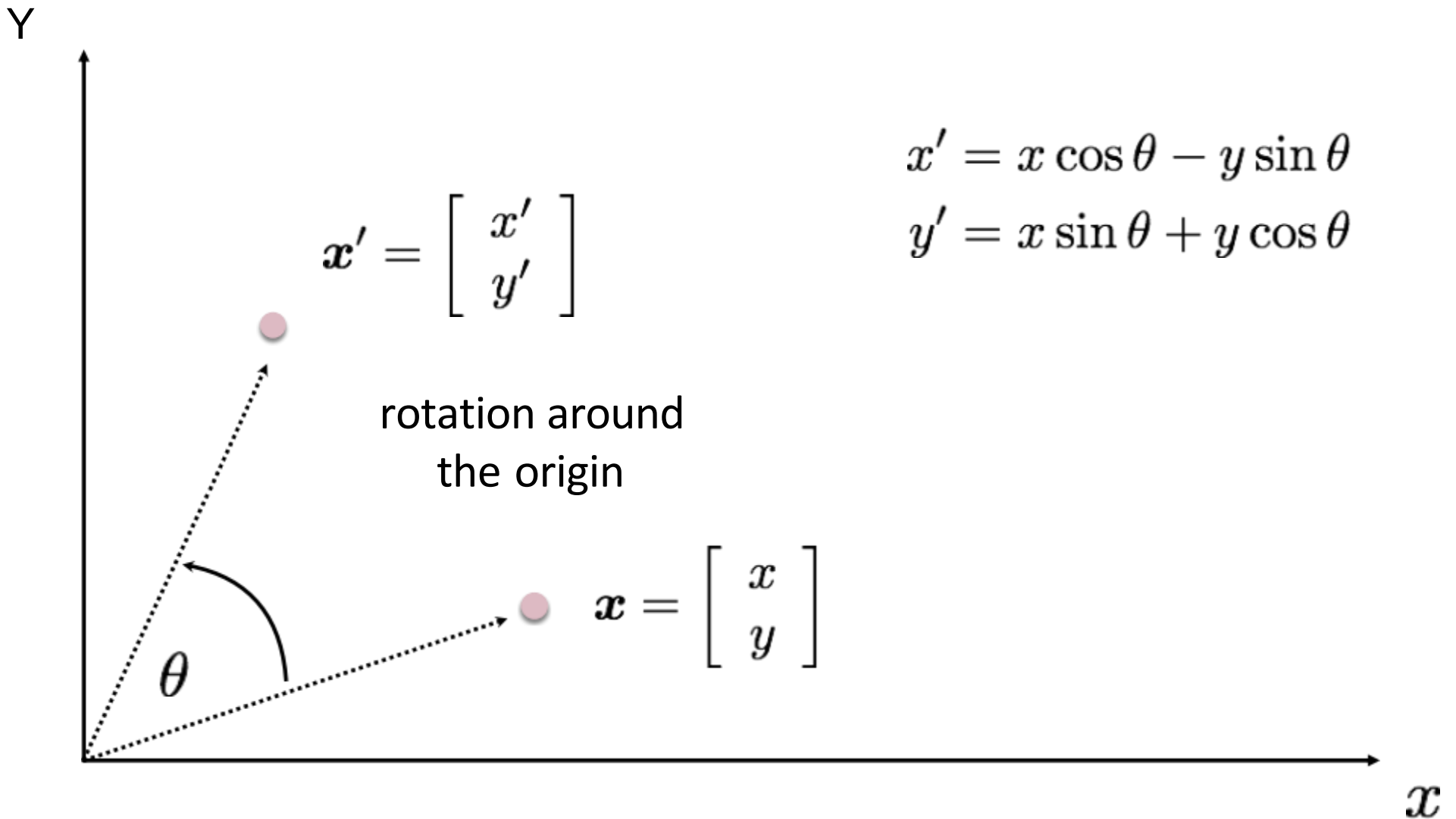
2D planar transformations



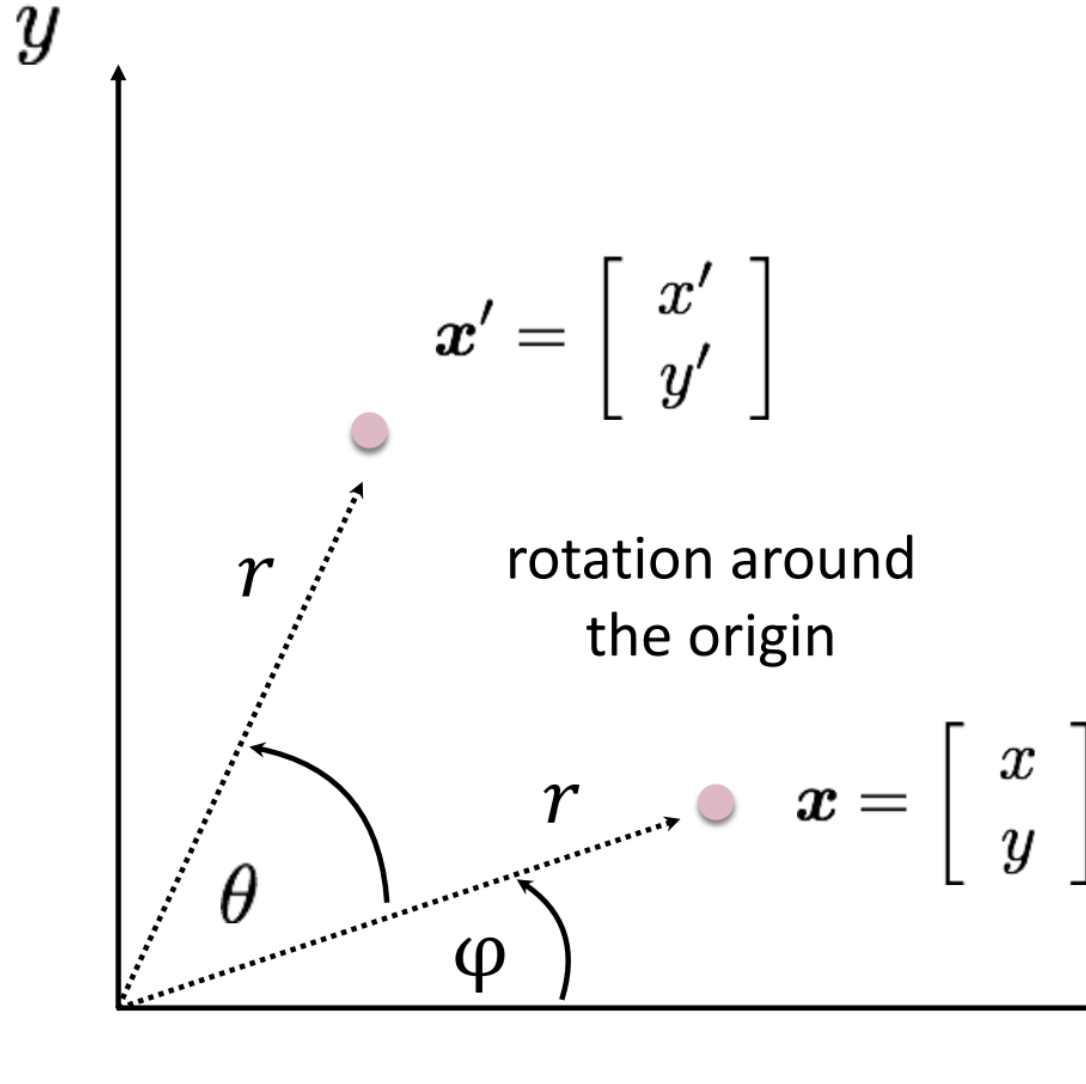
2D planar transformations



2D planar transformations



2D planar transformations



Polar coordinates...

$$x = r \cos(\phi)$$

$$y = r \sin(\phi)$$

$$x' = r \cos(\phi + \theta)$$

$$y' = r \sin(\phi + \theta)$$

Trigonometric Identity...

$$x' = r \cos(\phi) \cos(\theta) - r \sin(\phi) \sin(\theta)$$

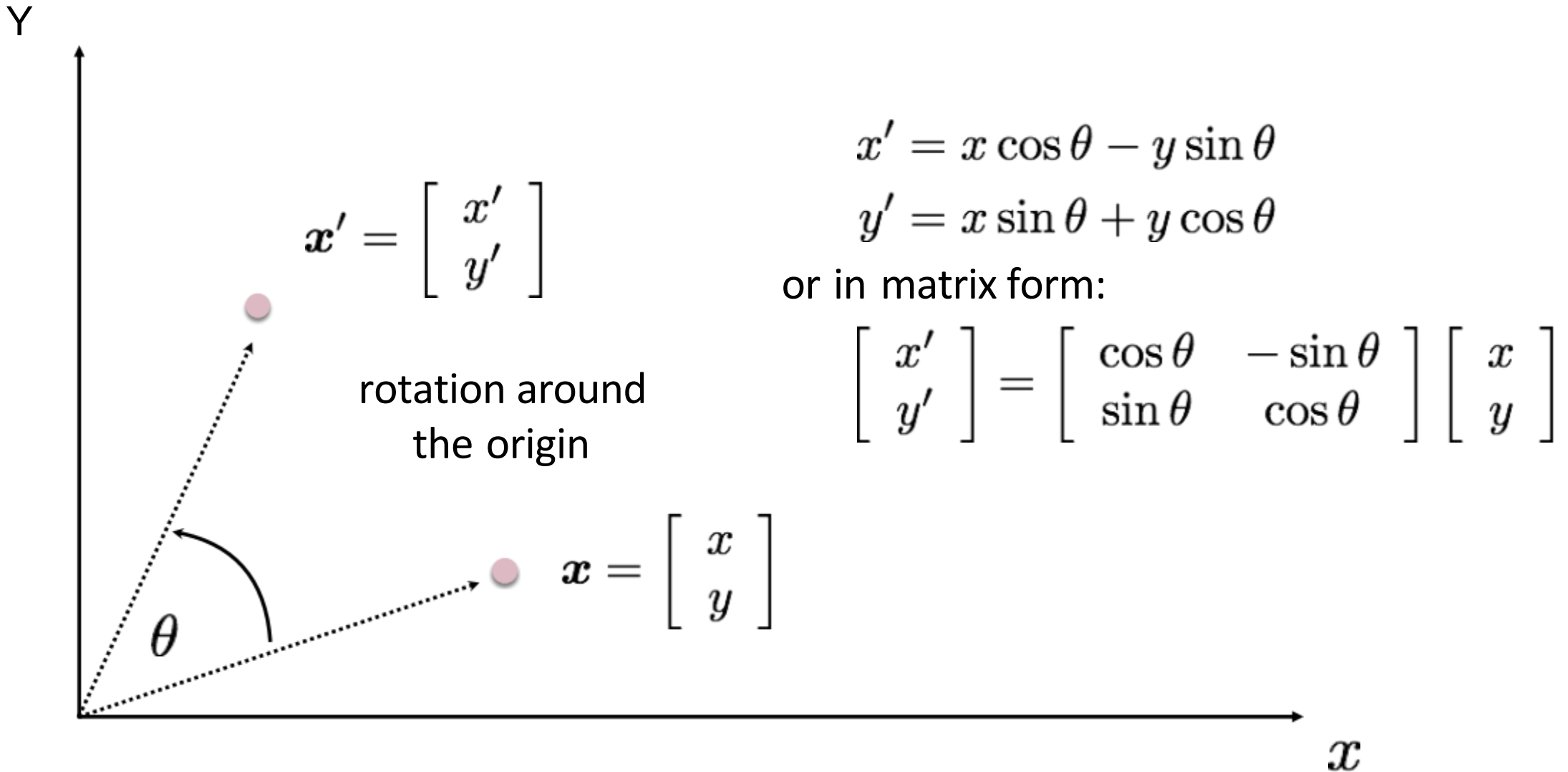
$$y' = r \sin(\phi) \cos(\theta) + r \cos(\phi) \sin(\theta)$$

Substitute...

$$x' = x \cos(\theta) - y \sin(\theta)$$

$$y' = x \sin(\theta) + y \cos(\theta)$$

2D planar transformations



2D planar and linear transformations

$$\boldsymbol{x}' = f(\boldsymbol{x}; p)$$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \boldsymbol{M} \begin{bmatrix} x \\ y \end{bmatrix}$$

parameters p

point $\boldsymbol{x}$

2D planar and linear transformations

Scale

$$\mathbf{M} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Flip across y

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Rotate

$$\mathbf{M} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Flip across origin

$$\mathbf{M} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

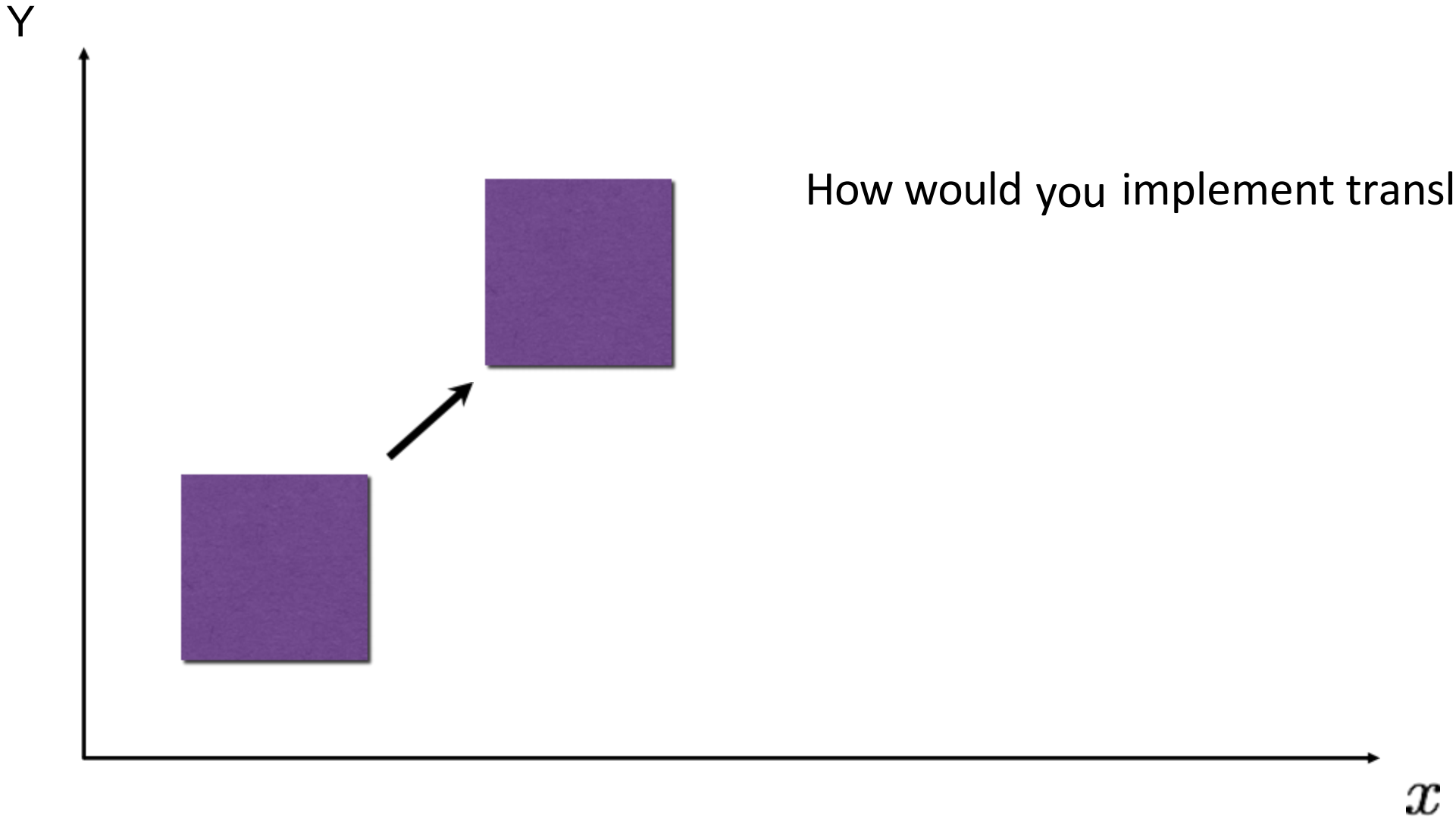
Shear

$$\mathbf{M} = \begin{bmatrix} 1 & s_x \\ s_y & 1 \end{bmatrix}$$

Identity

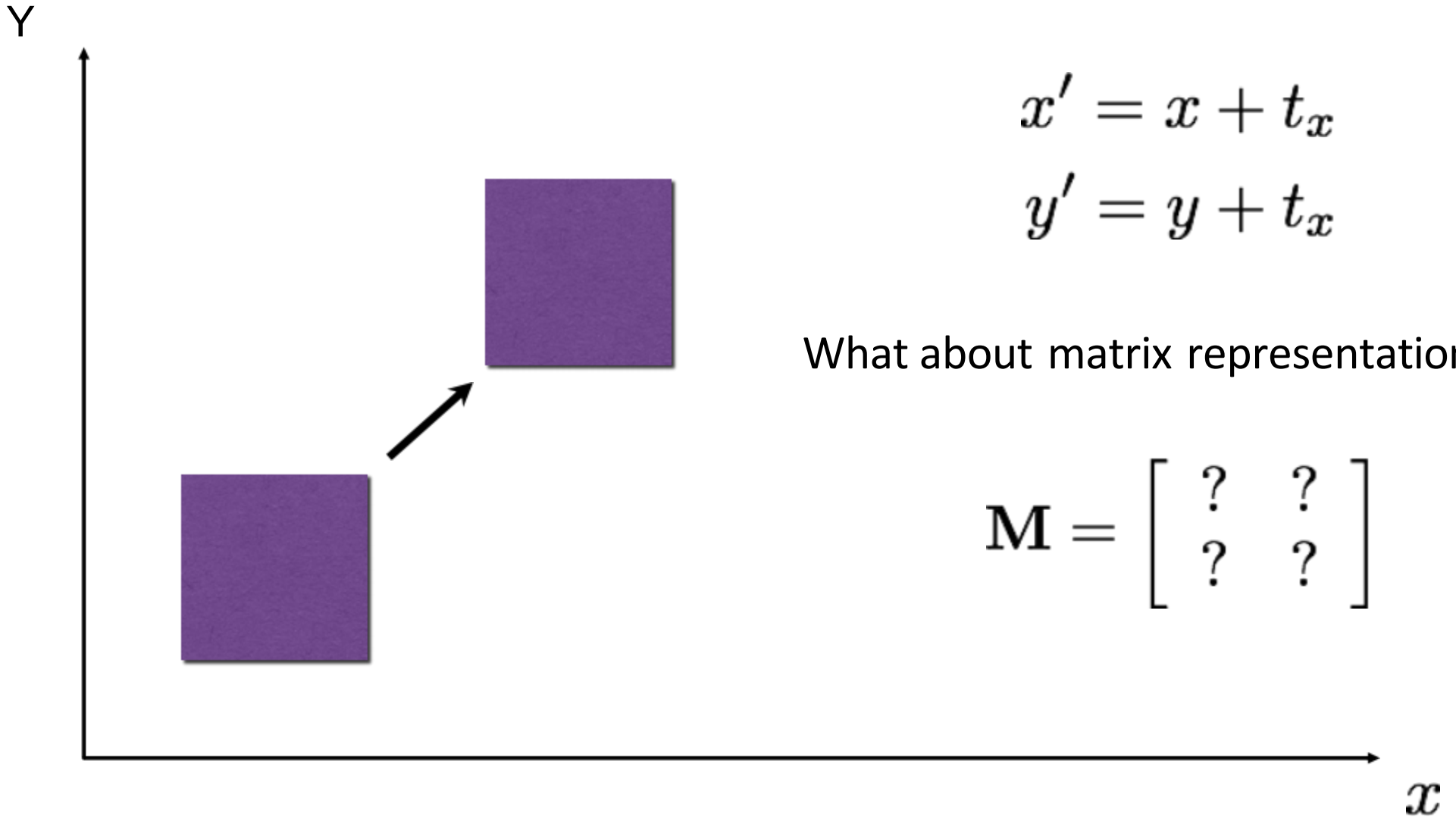
$$\mathbf{M} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2D translation

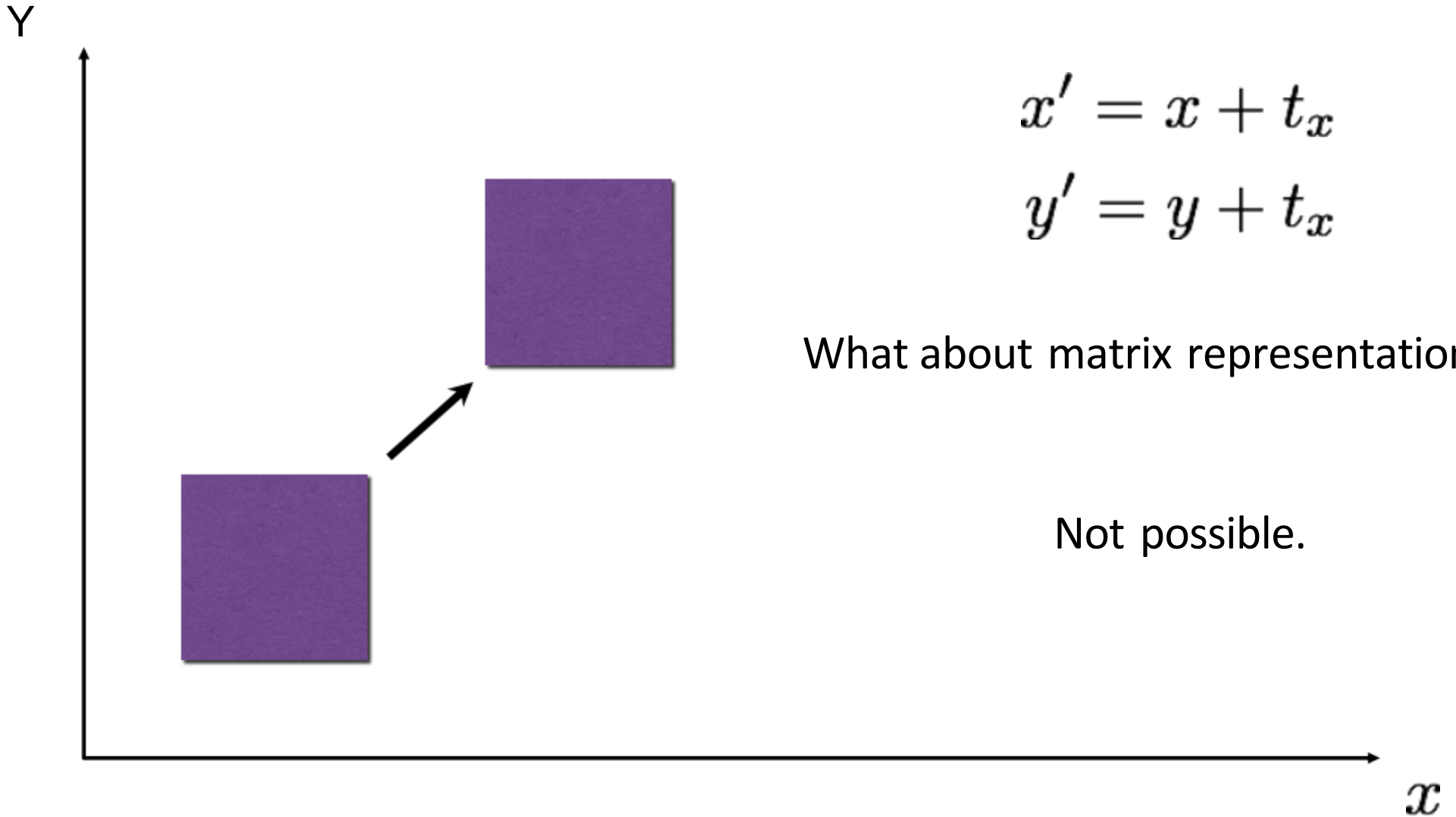


How would you implement translation?

2D translation



2D translation



Projective geometry

Homogeneous coordinates

heterogeneous
coordinates

homogeneous
coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

add a 1 here



- Represent 2D point with a 3D vector

Homogeneous coordinates

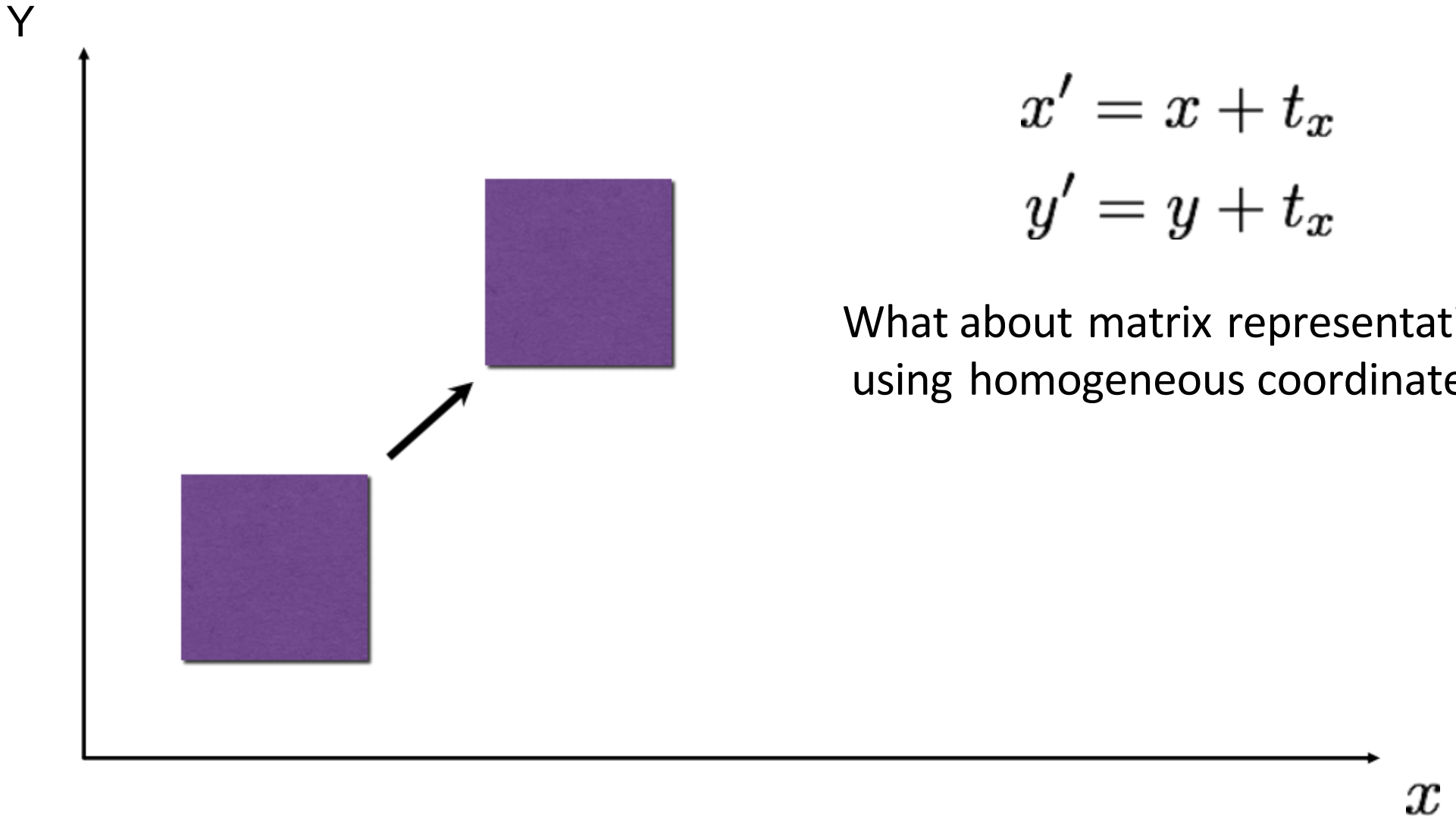
heterogeneous
coordinates

homogeneous
coordinates

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \stackrel{\text{def}}{=} \begin{bmatrix} ax \\ ay \\ a \end{bmatrix}$$

- Represent 2D point with a 3D vector
- 3D vectors are only defined up to scale

2D translation



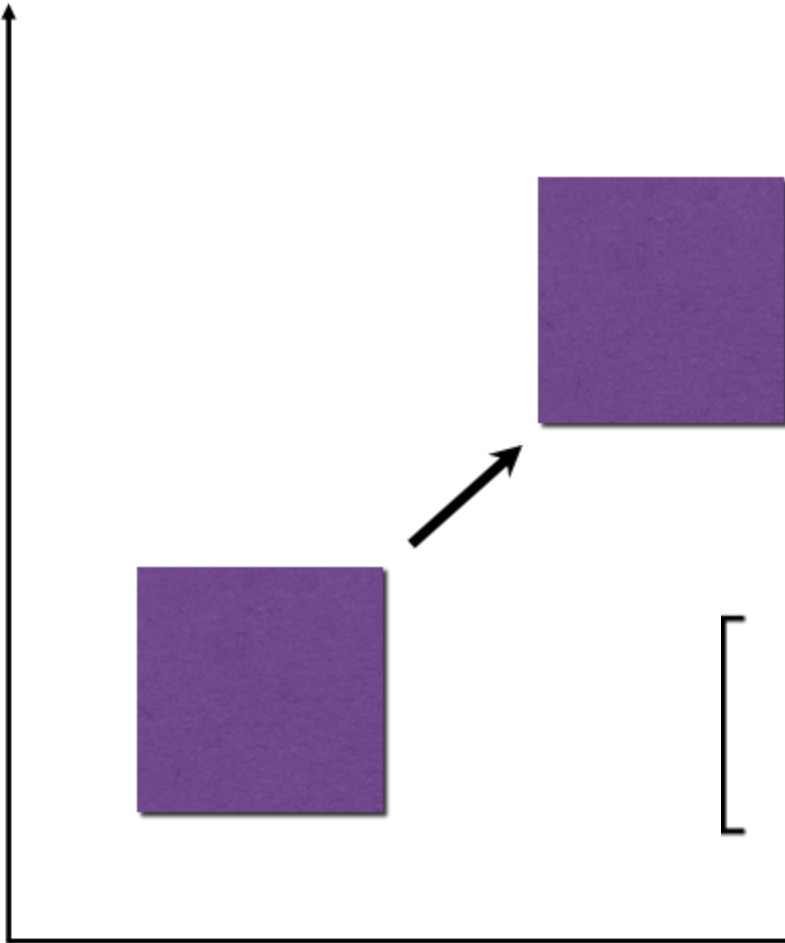
$$x' = x + t_x$$

$$y' = y + t_y$$

What about matrix representation
using homogeneous coordinates?

2D translation

Y



$$x' = x + t_x$$

$$y' = y + t_y$$

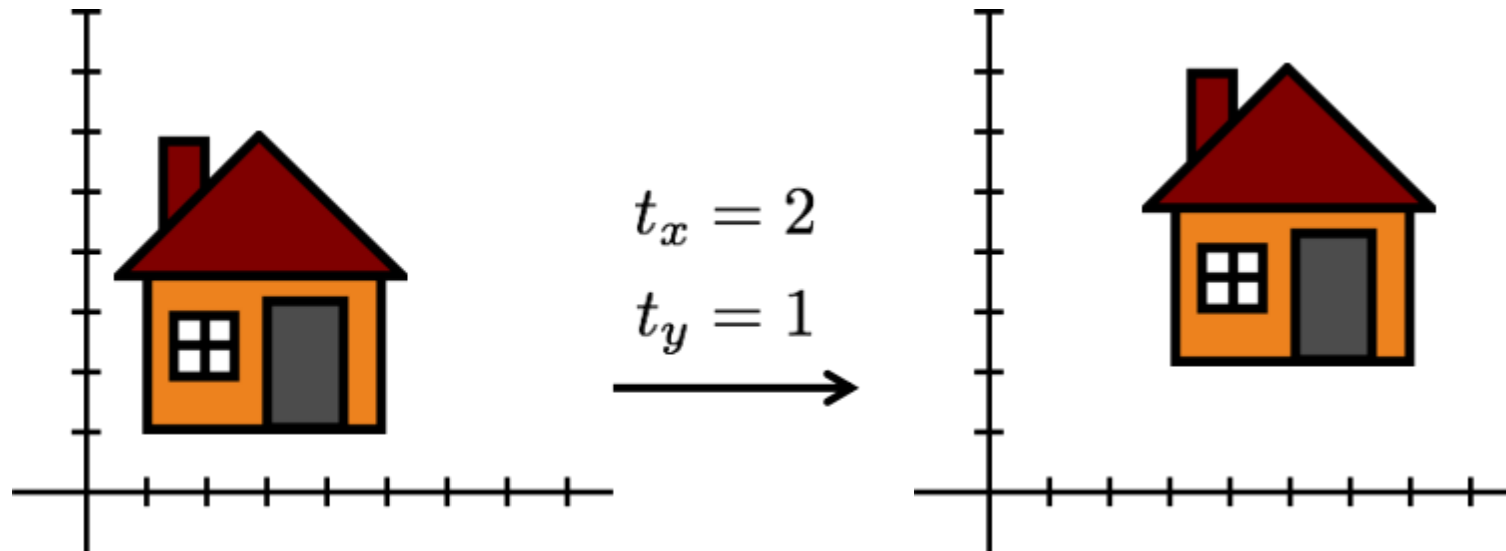
What about matrix representation
using homogeneous coordinates?

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \mathbf{M} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

x

2D translation using homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$



Homogeneous coordinates

Conversion:

- heterogeneous $\rightarrow$ homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- homogeneous $\rightarrow$ heterogeneous

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow \begin{bmatrix} x/w \\ y/w \end{bmatrix}$$

- scale invariance

$$\begin{bmatrix} x & y & w \end{bmatrix}^{\top} = \lambda \begin{bmatrix} x & y & w \end{bmatrix}^{\top}$$

Special points:

- point at infinity

$$\begin{bmatrix} x & y & 0 \end{bmatrix}$$

- undefined

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

Projective geometry

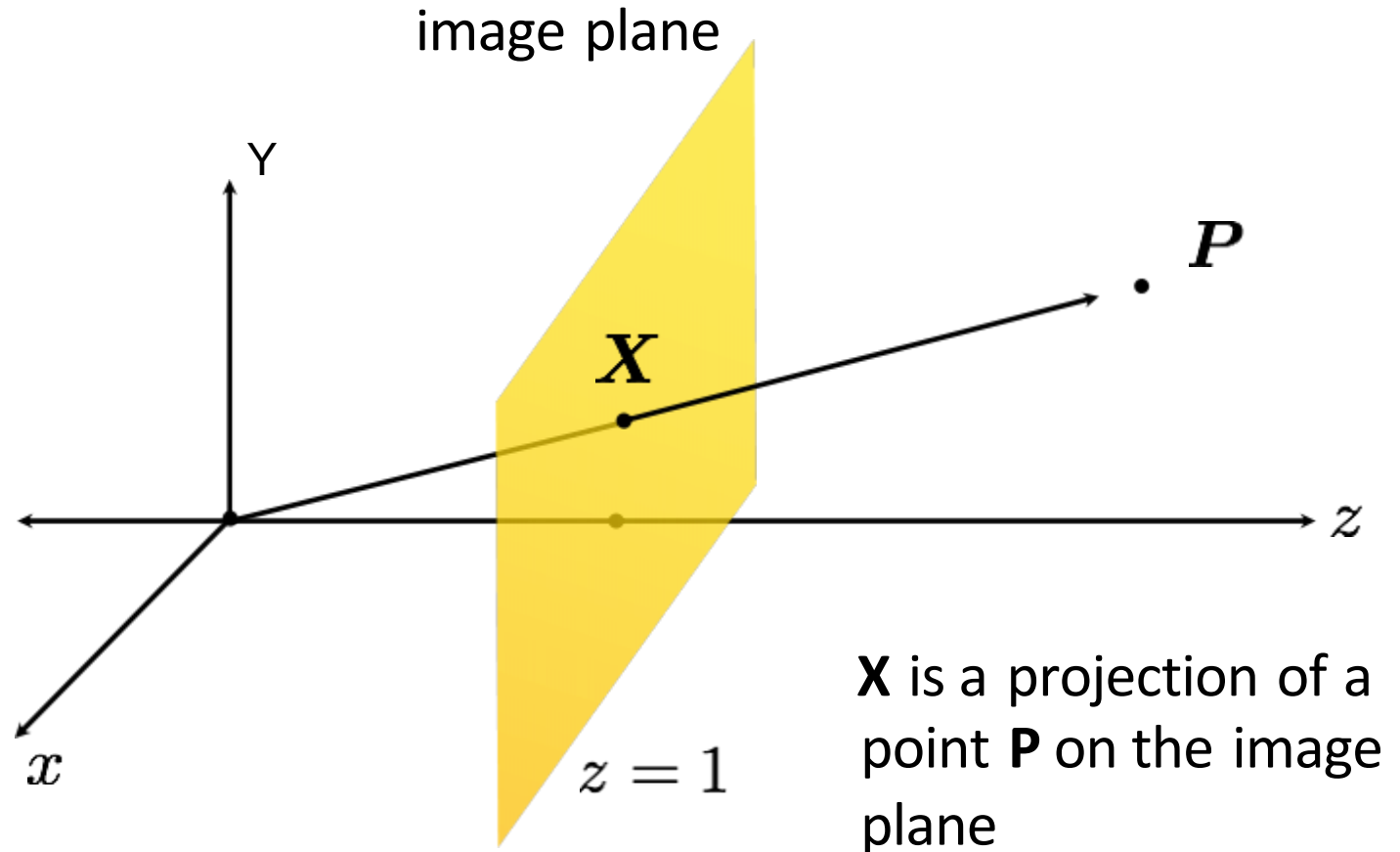
image point in
pixel coordinates

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$



image point in
homogeneous
coordinates

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Transformations in projective geometry

2D transformations in heterogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} ? \\ ? \\ ? \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} ? \\ ? \\ ? \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} ? \\ ? \\ ? \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

2D transformations in heterogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} ? \\ ? \\ ? \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} ? \\ ? \\ ? \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

2D transformations in heterogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} & & \\ & ? & \\ & & \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

2D transformations in heterogeneous coordinates

Re-write these transformations as 3x3 matrices:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

translation

$$\begin{bmatrix} \mathbf{x}' \\ \mathbf{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ 1 \end{bmatrix}$$

scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta_x & 0 \\ \beta_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

shearing

Matrix composition

Transformations can be combined by matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$\mathbf{p}' = \quad ? \quad ? \quad ? \quad \mathbf{p}$$

Matrix composition

Transformations can be combined by matrix multiplication:

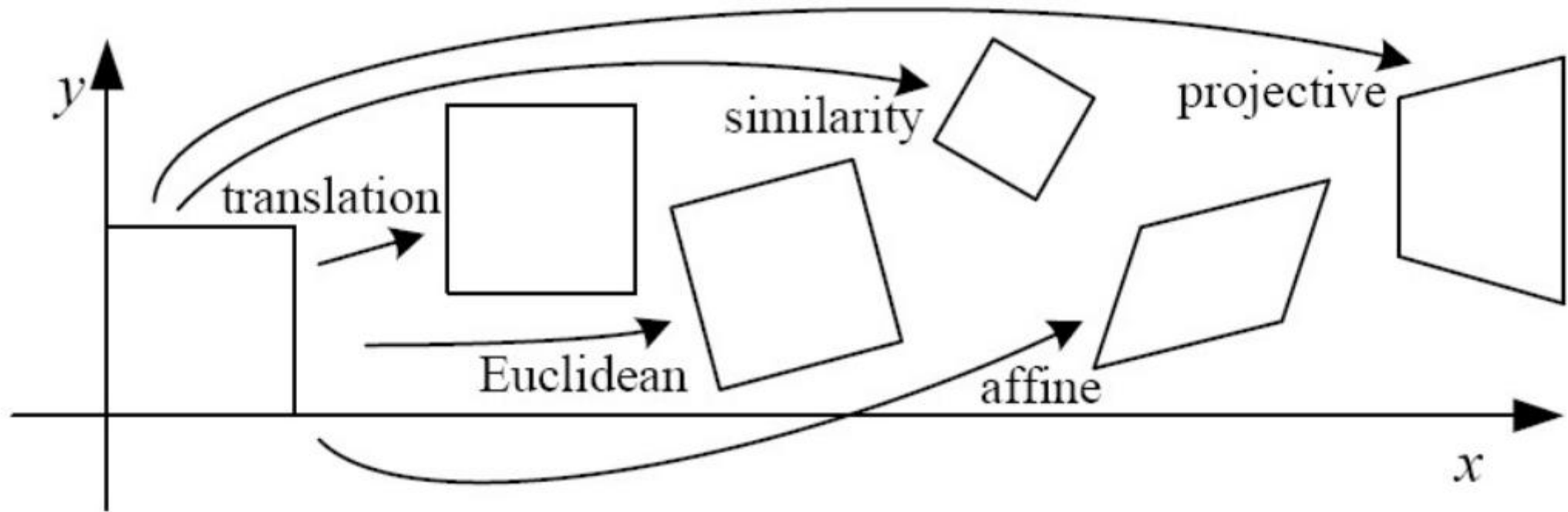
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$\mathbf{p}' = \text{translation}(t_x, t_y) \quad \text{rotation}(\theta) \quad \text{scale}(s, s) \quad \mathbf{p}$$

Does the multiplication order matter?

Classification of 2D transformations

Classification of 2D transformations



Classification of 2D transformations

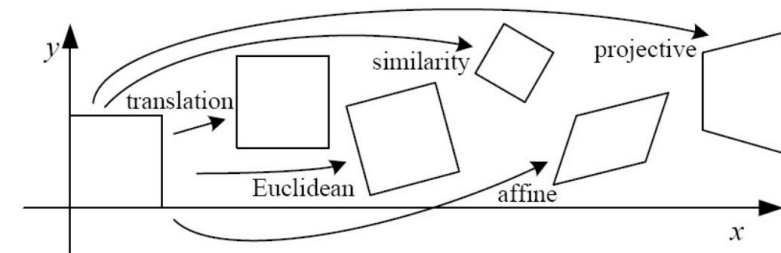
Name	Matrix	# D.O.F.
translation	$\begin{bmatrix} \mathbf{I} & & \mathbf{t} \end{bmatrix}$	?
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & & \mathbf{t} \end{bmatrix}$	?
similarity	$\begin{bmatrix} s\mathbf{R} & & \mathbf{t} \end{bmatrix}$	?
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}$	?
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}$	?

Classification of 2D transformations

Translation:

$$\begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix}$$

How many degrees of freedom?

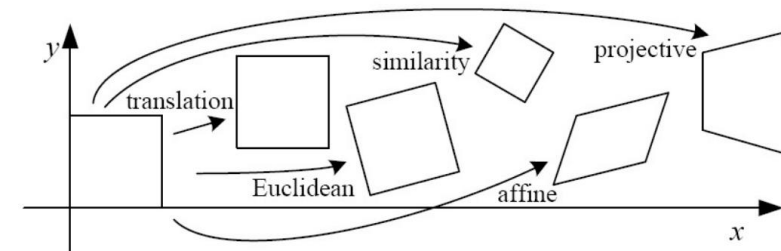


Classification of 2D transformations

Euclidean (rigid):
rotation + translation

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

Are there any values that are related?

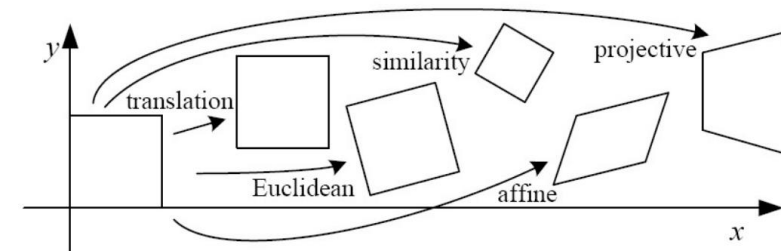


Classification of 2D transformations

Euclidean (rigid):
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & r_3 \\ \sin \theta & \cos \theta & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

How many degrees of freedom?

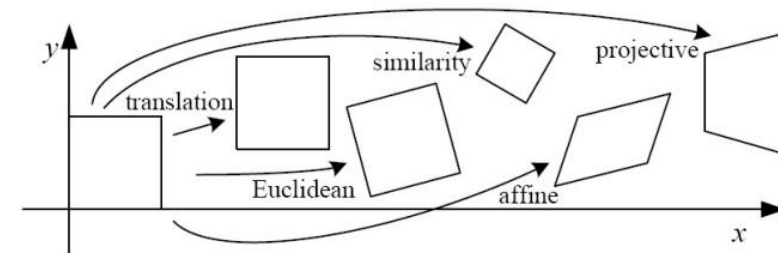


Classification of 2D transformations

which other matrix values
will change if this increases?

Euclidean (rigid):
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & r_3 \\ \sin \theta & \cos \theta & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$



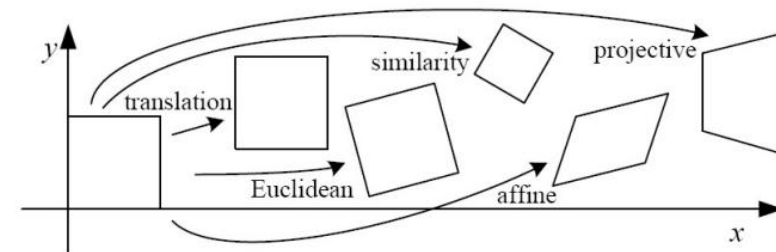
Classification of 2D transformations

what will happen to the
image if this increases?



Euclidean (rigid):
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & r_3 \\ \sin \theta & \cos \theta & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$



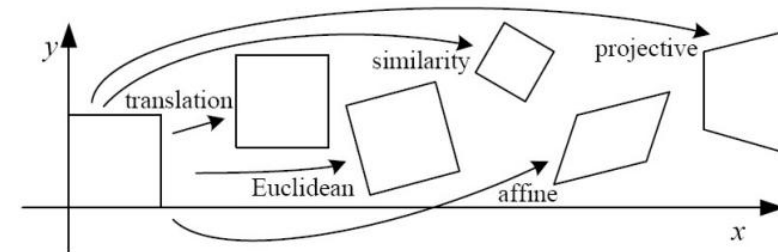
Classification of 2D transformations

what will happen to the
image if this increases?



Euclidean (rigid):
rotation + translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & r_3 \\ \sin \theta & \cos \theta & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

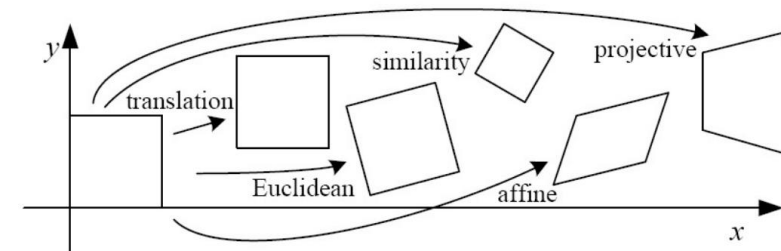


Classification of 2D transformations

Similarity:
uniform scaling + rotation
+ translation

$$\begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

Are there any values that are related?



Classification of 2D transformations

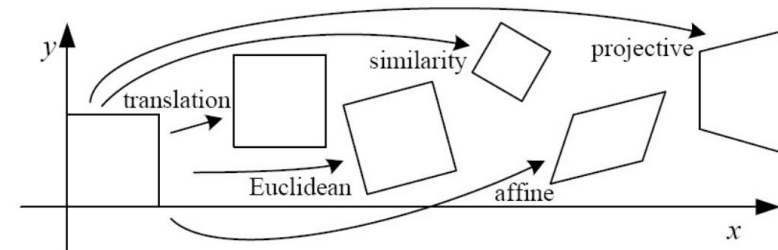
multiply these four by scale s

!

Similarity:
uniform scaling + rotation
+ translation

$$\begin{bmatrix} \cos \theta & -\sin \theta & r_3 \\ \sin \theta & \cos \theta & r_6 \\ 0 & 0 & 1 \end{bmatrix}$$

How many degrees of freedom?

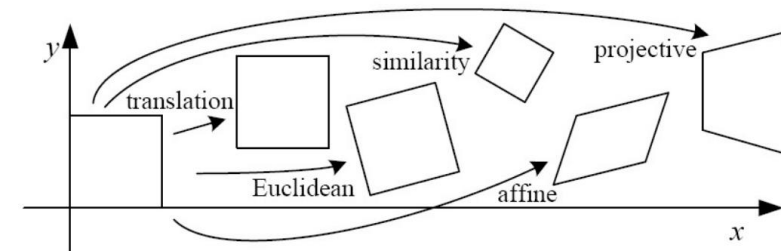


Classification of 2D transformations

Affine transform:
uniform scaling + shearing
+ rotation + translation

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ 0 & 0 & 1 \end{bmatrix}$$

Are there any values that are related?



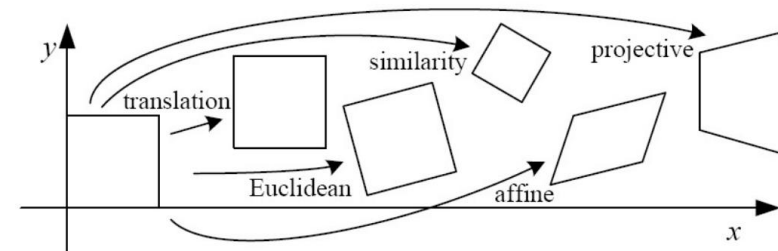
Classification of 2D transformations

Affine transform:
uniform scaling + shearing
+ rotation + translation

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ 0 & 0 & 1 \end{bmatrix}$$

Are there any values that are related?

$$\begin{matrix} \text{similarity} \\ \begin{bmatrix} sr_1 & sr_2 \\ sr_3 & sr_4 \end{bmatrix} \end{matrix} \begin{matrix} \text{shear} \\ \begin{bmatrix} 1 & h_1 \\ h_2 & 1 \end{bmatrix} \end{matrix} = \begin{bmatrix} sr_1 + h_2 sr_2 & sr_2 + h_1 sr_1 \\ sr_3 + h_2 sr_4 & sr_4 + h_1 sr_3 \end{bmatrix}$$



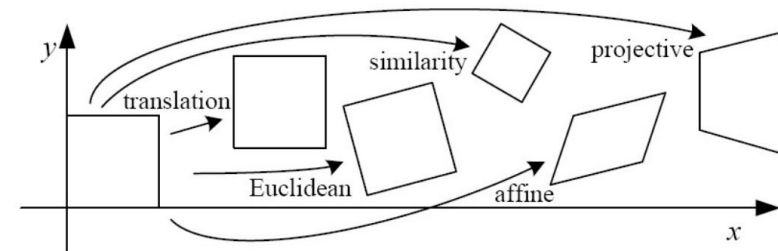
Classification of 2D transformations

Affine transform:
uniform scaling + shearing
+ rotation + translation

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ 0 & 0 & 1 \end{bmatrix}$$

How many degrees of freedom?

$$\begin{array}{cc} \text{similarity} & \text{shear} \\ \begin{bmatrix} sr_1 & sr_2 \\ sr_3 & sr_4 \end{bmatrix} & \begin{bmatrix} 1 & h_1 \\ h_2 & 1 \end{bmatrix} \end{array} = \begin{bmatrix} sr_1 + h_2 sr_2 & sr_2 + h_1 sr_1 \\ sr_3 + h_2 sr_4 & sr_4 + h_1 sr_3 \end{bmatrix}$$



Affine transformations

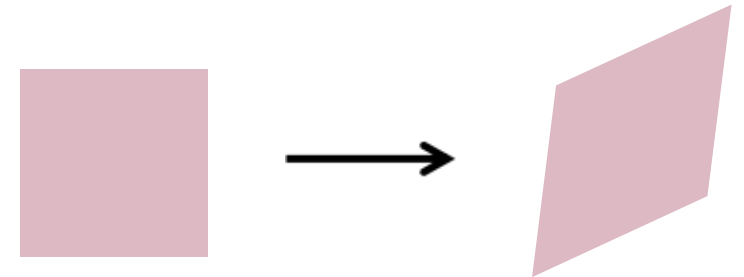
Affine transformations are combinations of

- arbitrary (4-DOF) linear transformations; and
- translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of affine transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines map to parallel lines
- ratios are preserved
- compositions of affine transforms are also affine transforms



Does the last coordinate w ever change?

Affine transformations

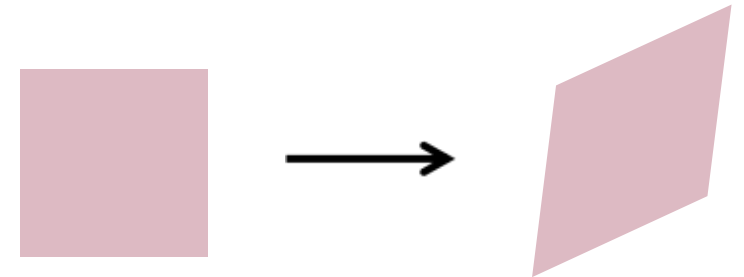
Affine transformations are combinations of

- arbitrary (4-DOF) linear transformations; and
- translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of affine transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines map to parallel lines
- ratios are preserved
- compositions of affine transforms are also affine transforms



Nope! But what does that mean?

How to interpret affine transformations here?

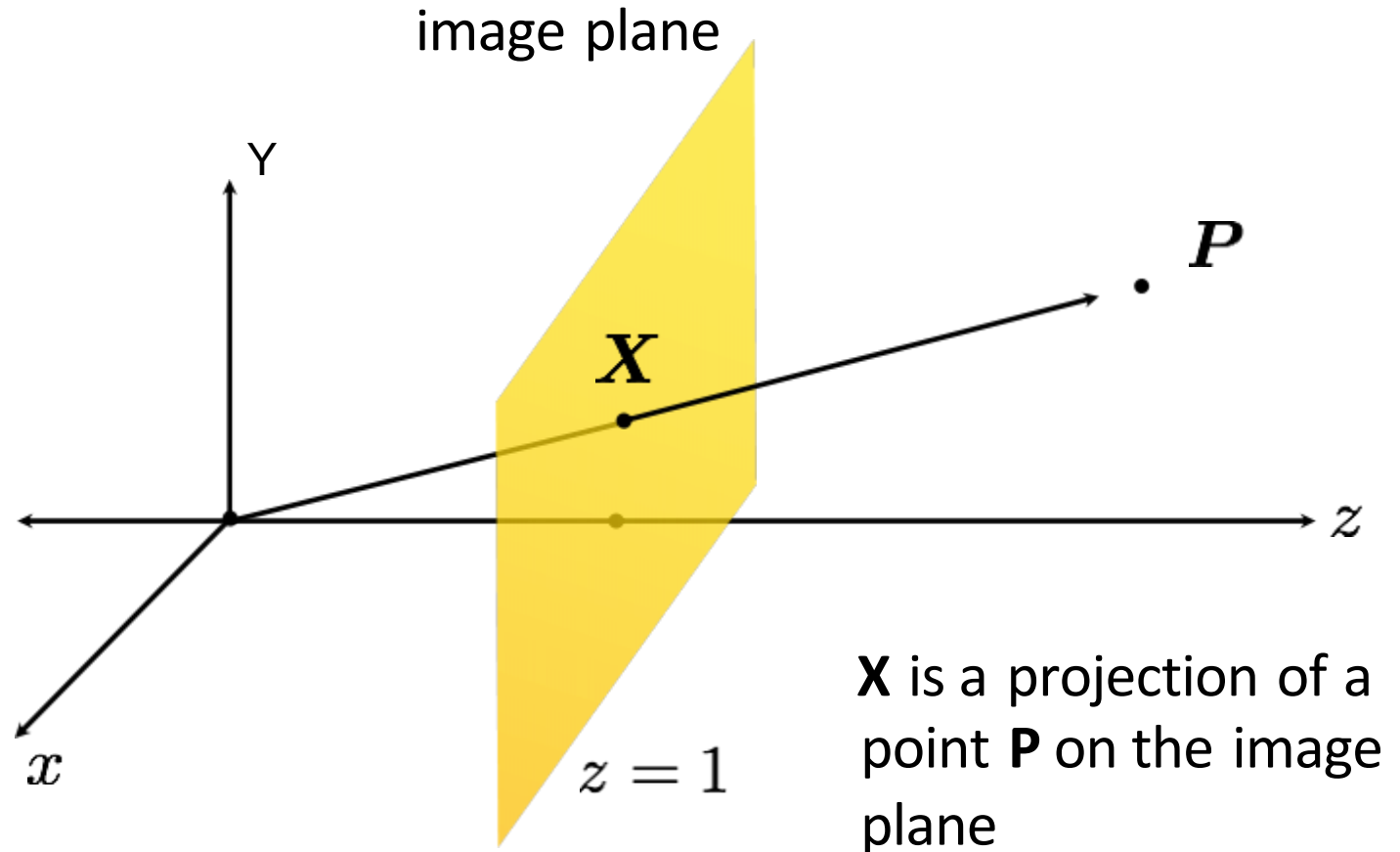
image point in
pixel coordinates

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$



image point in
heterogeneous
coordinates

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Projective transformations (aka homographies)

Projective transformations are combinations of

- affine transformations; and
- projective wraps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

How many degrees of freedom?

Properties of projective transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines do not necessarily map to parallel lines
- ratios are not necessarily preserved
- compositions of projective transforms are also projective transforms



Projective transformations (aka homographies)

Projective transformations are combinations of

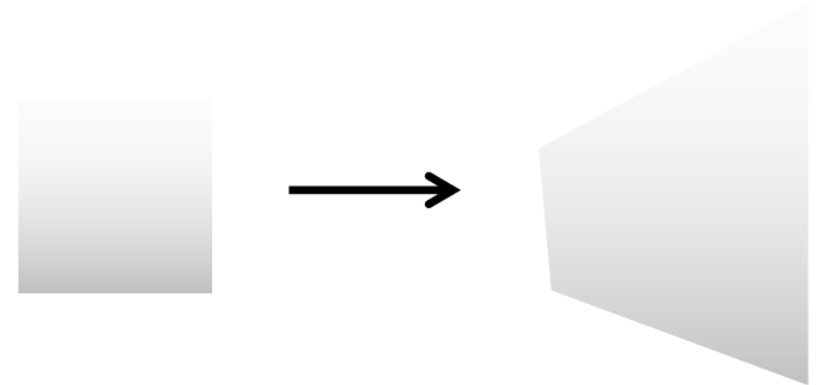
- affine transformations; and
- projective wraps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of projective transformations:

- origin does not necessarily map to origin
- lines map to lines
- parallel lines do not necessarily map to parallel lines
- ratios are not necessarily preserved
- compositions of projective transforms are also projective transforms

8 DOF: vectors (and therefore matrices) are defined up to scale)



How to interpret projective transformations here?

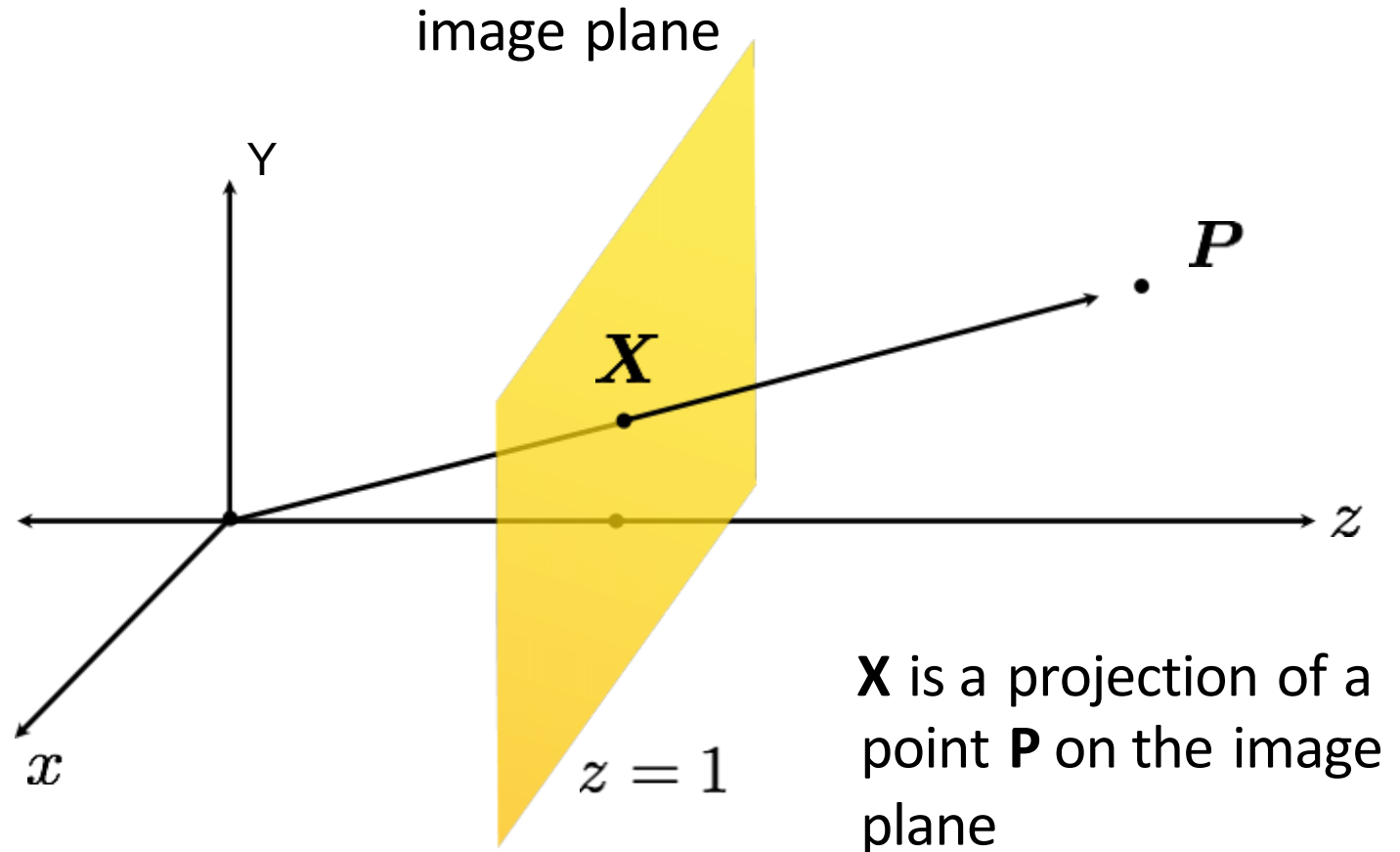
image point in
pixel coordinates

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$



image point in
heterogeneous
coordinates

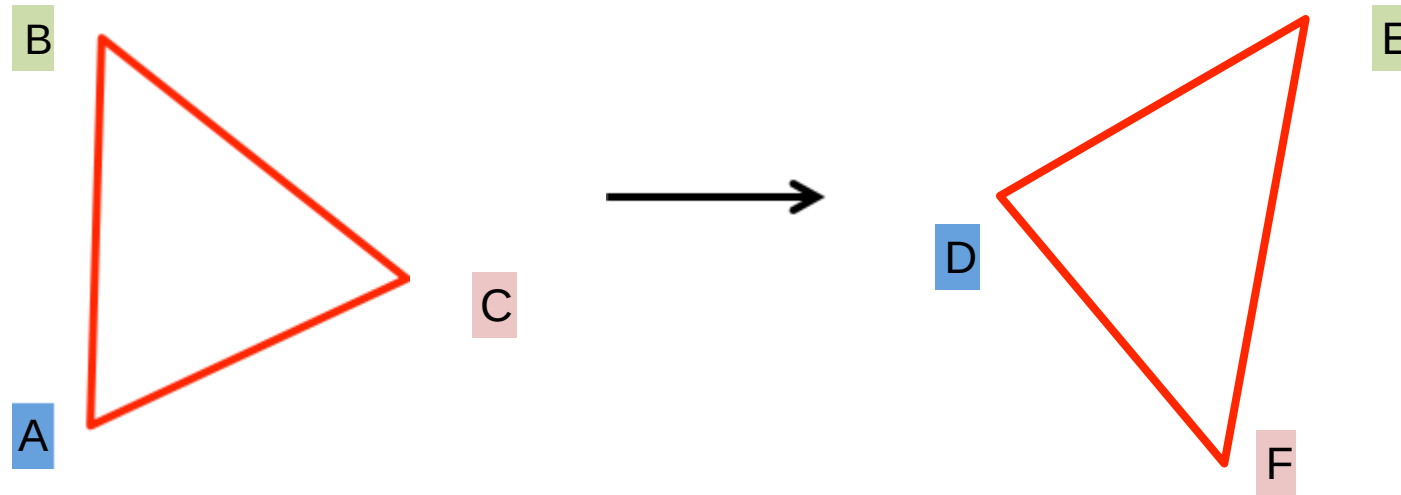
$$\mathbf{X} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Determining unknown (affine) 2D transformations

Determining unknown transformations

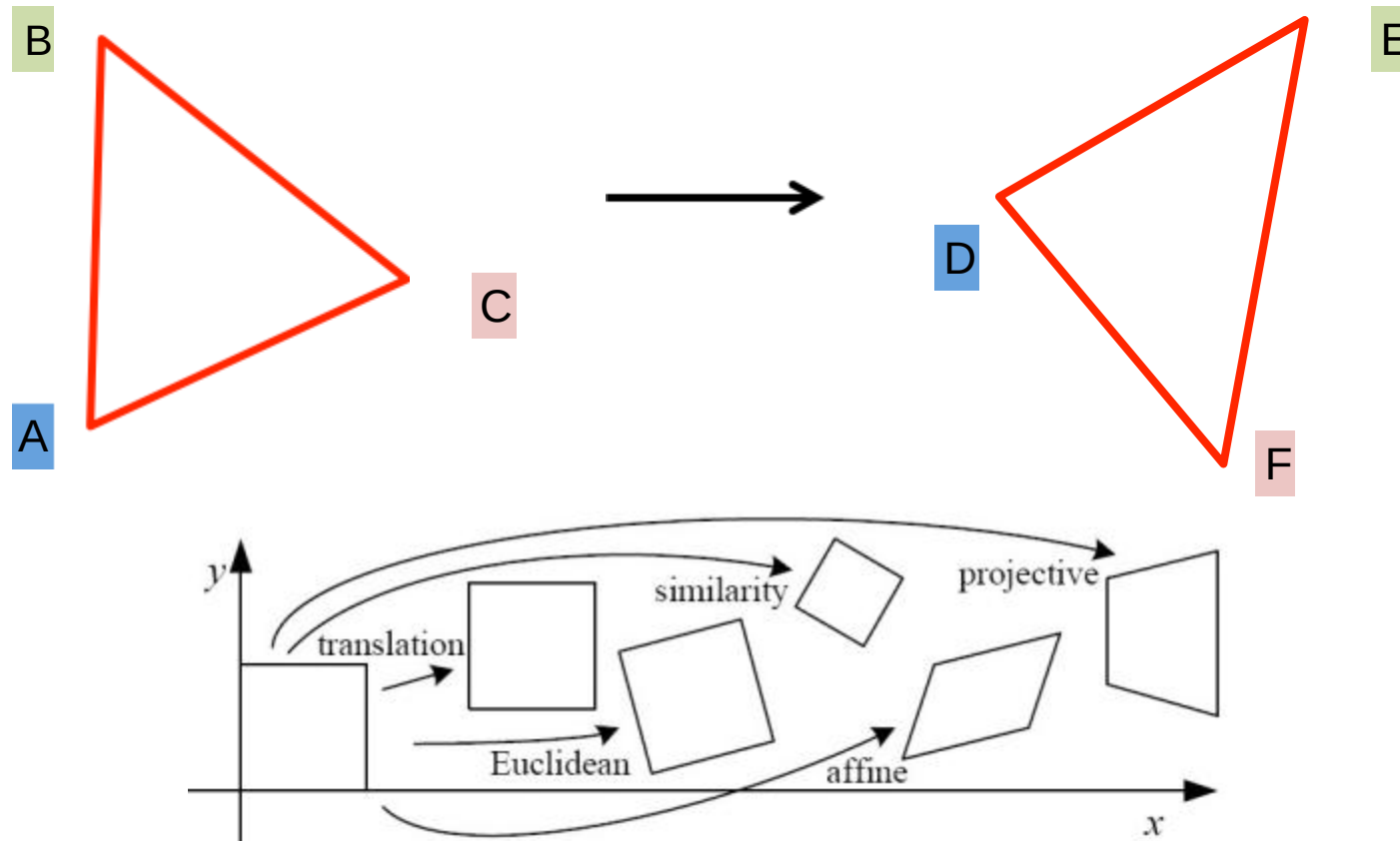
Suppose we have two triangles: ABC and DEF.



Determining unknown transformations

Suppose we have two triangles: ABC and DEF.

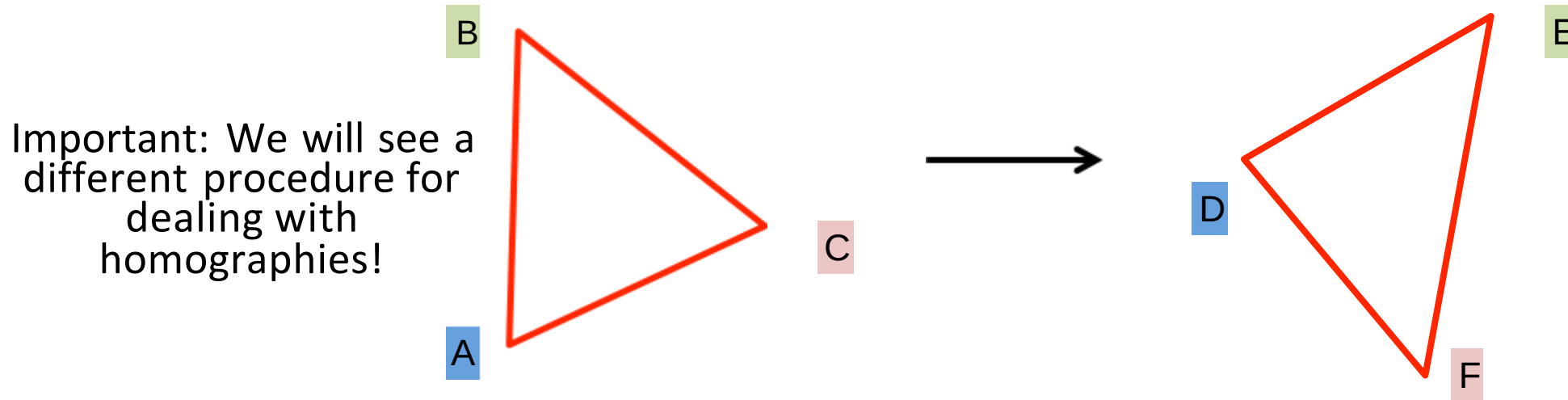
- What type of transformation will map A to D, B to E, and C to F?



Determining unknown transformations

Suppose we have two triangles: ABC and DEF.

- What type of transformation will map A to D, B to E, and C to F?
- How do we determine the unknown parameters?



Affine transform:
uniform scaling + shearing
+ rotation + translation

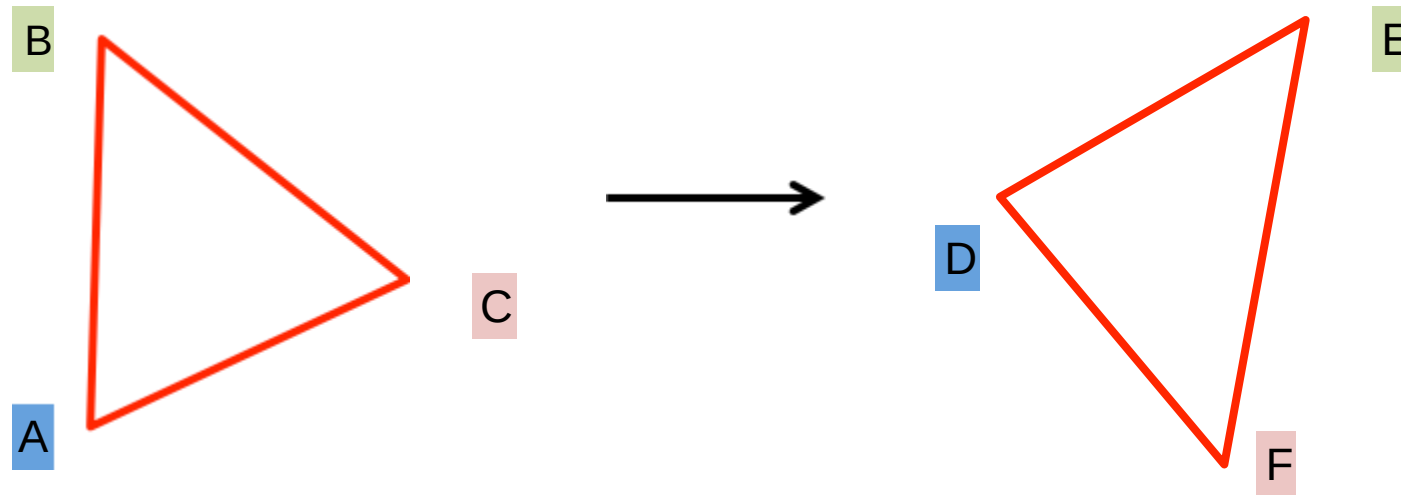
$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ 0 & 0 & 1 \end{bmatrix}$$

How many degrees of freedom do we have?

Determining unknown transformations

Suppose we have two triangles: ABC and DEF.

- What type of transformation will map A to D, B to E, and C to F?
- How do we determine the unknown parameters?



unknowns

$$x' = Mx$$

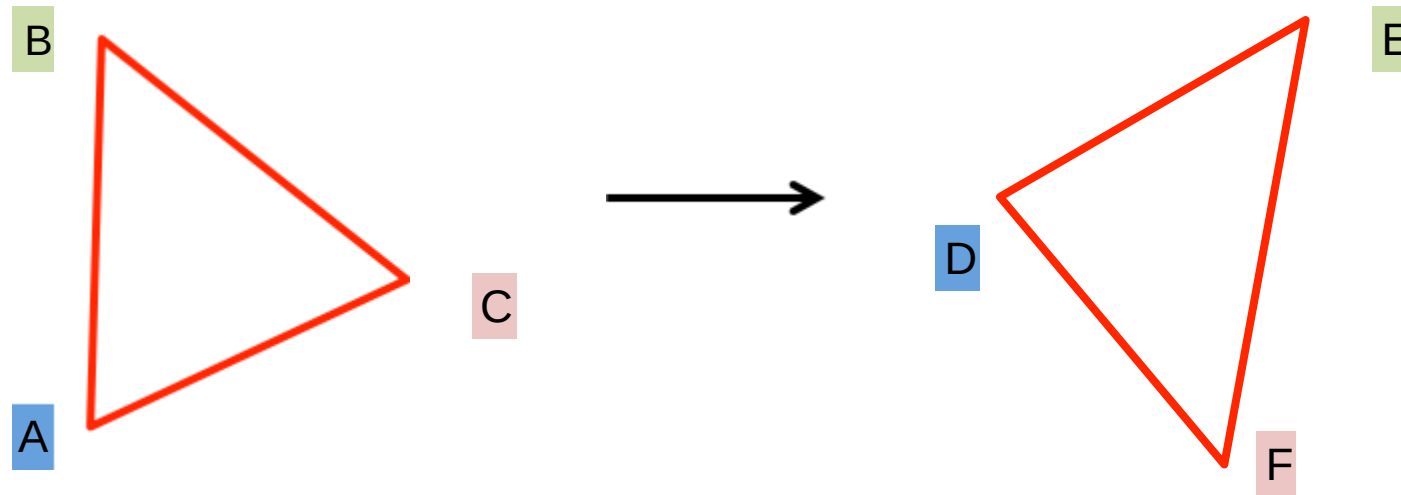
point correspondences

- One point correspondence gives how many equations?
- How many point correspondences do we need?

Determining unknown transformations

Suppose we have two triangles: ABC and DEF.

- What type of transformation will map A to D, B to E, and C to F?
- How do we determine the unknown parameters?

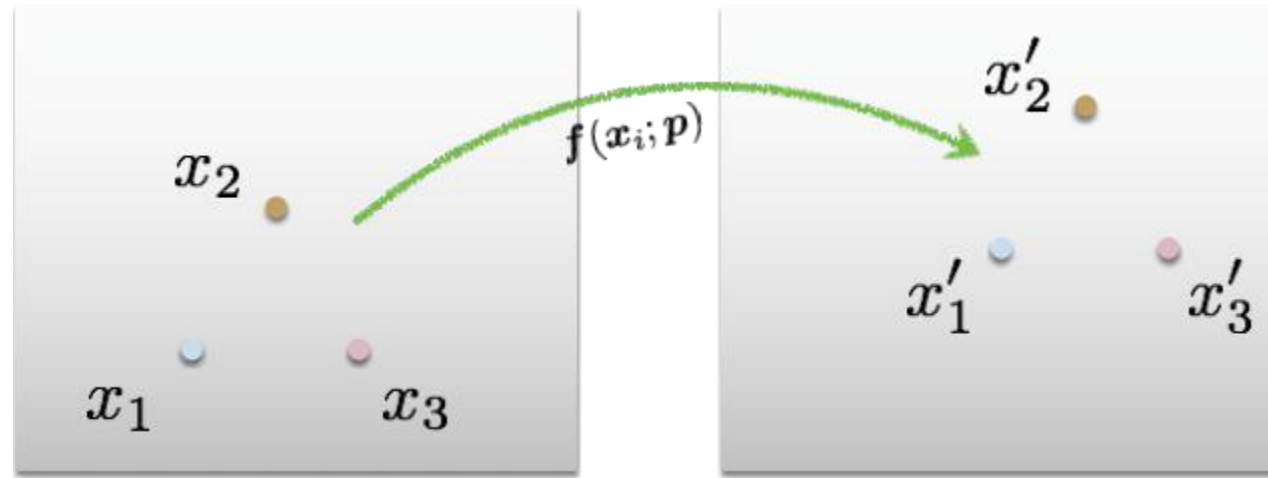


unknowns

$$x' = Mx$$

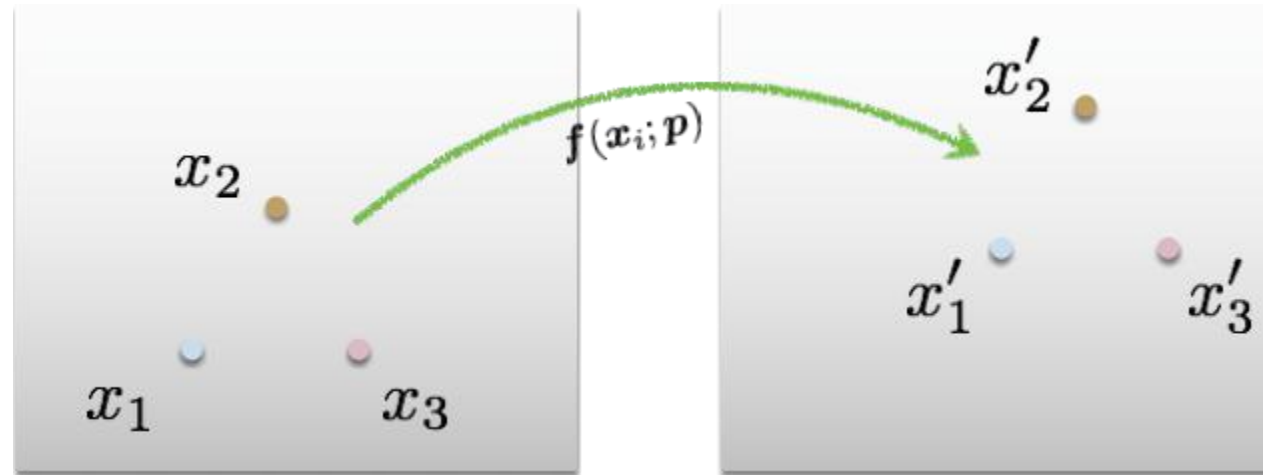
point correspondences

How do we solve this for **M**?



Least Squares Error

$$E_{\text{LS}} = \sum_i \|\mathbf{f}(\mathbf{x}_i; \mathbf{p}) - \mathbf{x}'_i\|^2$$



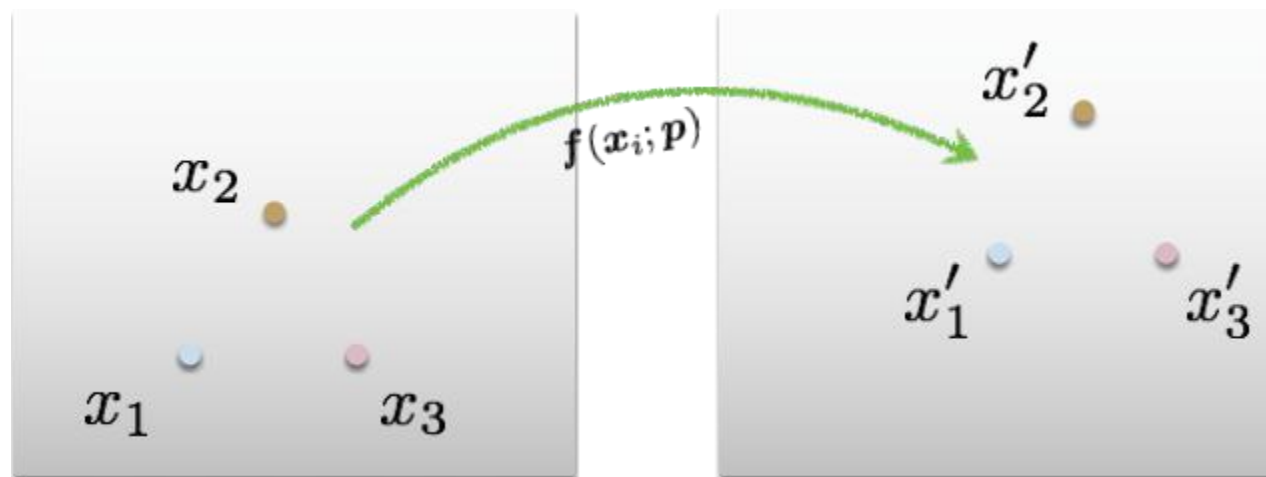
Least Squares Error

$$E_{\text{LS}} = \sum_i \left\| \mathbf{f}(x_i; \mathbf{p}) - \mathbf{x}'_i \right\|^2$$

What is this?

What is this?

What is this?

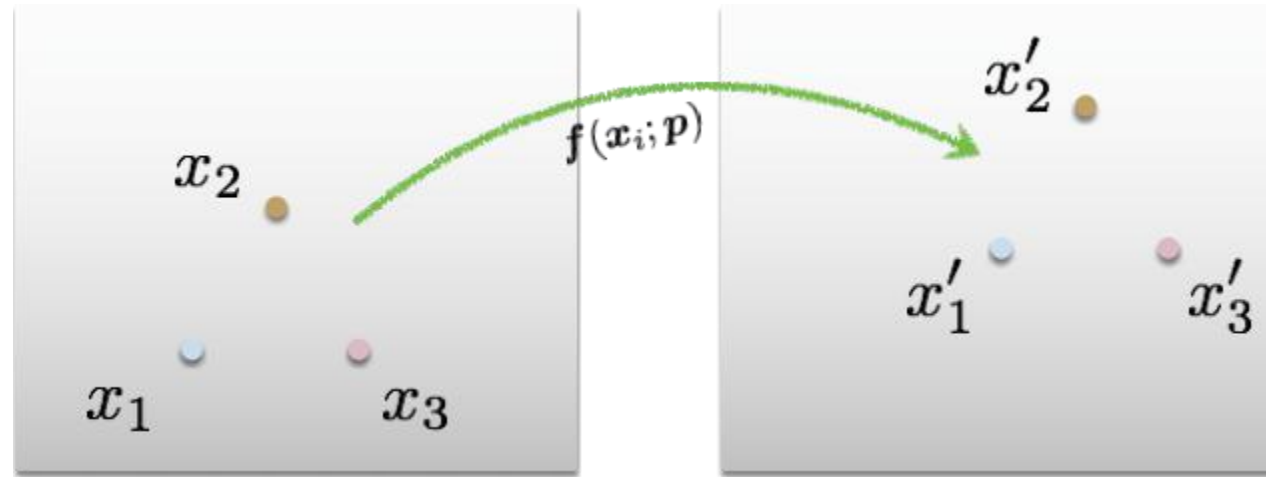


$$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

Least Squares Error

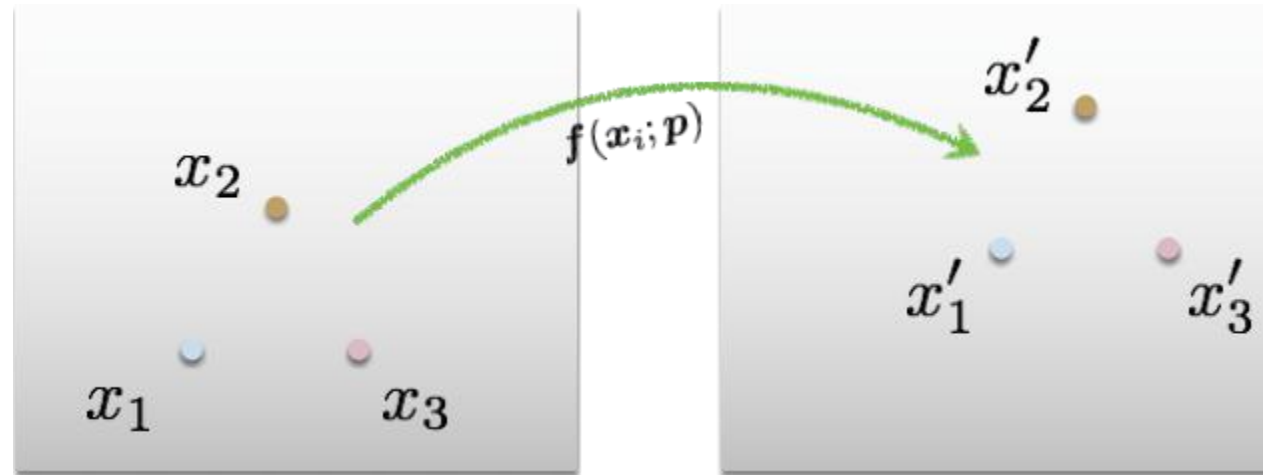
$$E_{\text{LS}} = \sum_i \left\| \underset{\substack{\uparrow \\ \text{predicted} \\ \text{location}}}{f(x_i; p)} - \underset{\substack{\uparrow \\ \text{measured} \\ \text{location}}}{x'_i} \right\| \overset{\substack{\uparrow \\ \text{squared!}}}{^2}$$

Euclidean (L2) norm



Least Squares Error

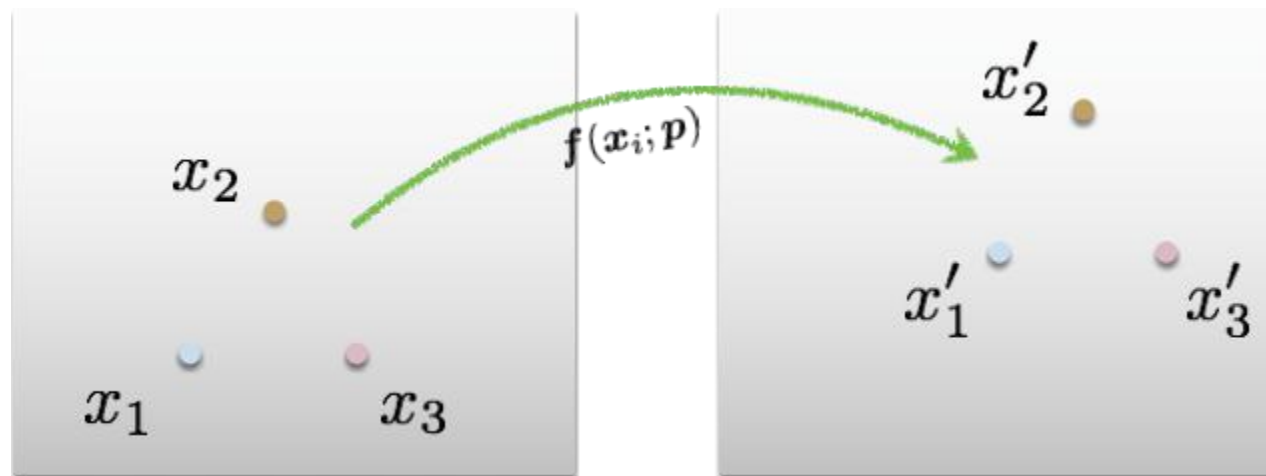
$$E_{\text{LS}} = \sum_i \underbrace{\| \mathbf{f}(\mathbf{x}_i; \mathbf{p}) - \mathbf{x}'_i \|^2}_{\text{Residual (projection error)}}$$



Least Squares Error

$$E_{\text{LS}} = \sum_i \|\mathbf{f}(\mathbf{x}_i; \mathbf{p}) - \mathbf{x}'_i\|^2$$

What do we want to optimize?



Find parameters that minimize squared error

$$\hat{\mathbf{p}} = \arg \min_{\mathbf{p}} \sum_i \|\mathbf{f}(\mathbf{x}_i; \mathbf{p}) - \mathbf{x}'_i\|^2$$

General form of linear least squares

(**Warning:** change of notation. $\mathbf{x}$ is a vector of parameters!)

$$\begin{aligned} E_{\text{LLS}} &= \sum_i |\mathbf{a}_i \mathbf{x} - \mathbf{b}_i|^2 \\ &= \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 \quad (\text{matrix form}) \end{aligned}$$

Determining unknown transformations

Affine transformation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Why can we drop
the last line?

Vectorize transformation
parameters:

$$\begin{bmatrix} x' \\ y' \\ x' \\ y' \\ \vdots \\ x' \\ y' \end{bmatrix} = \begin{bmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \\ x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \\ \vdots & & & \vdots & & \\ x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \\ p_6 \end{bmatrix}$$

Stack equations from point
correspondences:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{bmatrix}$$

$\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{4.5cm}}$ $\underbrace{\hspace{1.5cm}}$

b

A

x

Notation in system form:

General form of linear least squares

(**Warning:** change of notation. $\mathbf{x}$ is a vector of parameters!)

$$\begin{aligned} E_{\text{LLS}} &= \sum_i |\mathbf{a}_i \mathbf{x} - \mathbf{b}_i|^2 \\ &= \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 \quad (\text{matrix form}) \end{aligned}$$

This function is quadratic.

How do you find the root of a quadratic?

Solving the linear system

Convert the system to a linear least-squares problem:

$$E_{\text{LLS}} = \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$$

Expand the error:

$$E_{\text{LLS}} = \mathbf{x}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{x} - 2\mathbf{x}^\top (\mathbf{A}^\top \mathbf{b}) + \|\mathbf{b}\|^2$$

Minimize the error:

Set derivative to 0 $(\mathbf{A}^\top \mathbf{A})\mathbf{x} = \mathbf{A}^\top \mathbf{b}$

Solve for $\mathbf{x}$ $\mathbf{x} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$ ←

In Python:

```
x = numpy.linalg.  
    solve(A, b)
```

Note: You almost never want to compute the inverse of a matrix.

Linear least squares estimation only works when the transform function is ?

Linear least squares estimation only works when the transform function is **linear! (duh)**

Also doesn't deal well with outliers